

Búsqueda Inteligente de Objetos en Simulación

Estado actual del proyecto y plan de implementación

Detección (YOLOE) + Búsqueda (VLMaps) + Navegación (Habitat-Sim)

Extensión: ROS 1 Noetic + Toyota HSR

Mario García Blasco

8 de junio de 2026

Índice

1. Visión general del proyecto	3
1.1. Problema	3
1.2. Hipótesis	3
1.3. Los tres niveles de conocimiento	3
1.4. Idea clave: VLMaps estrecha la búsqueda, YOLOE confirma	3
1.5. Etiquetado de habitaciones	3
2. Estado actual del proyecto (15 mayo 2026)	3
2.1. Infraestructura — COMPLETADA	3
2.2. Navegación interactiva — COMPLETADA	4
2.3. Mapper LLM — COMPLETADO	4
2.4. Siguiente extensión semántica antes de ROS — objetos no-VLMap abiertos . . .	4
2.5. YOLOE como verificador — COMPLETADO	5
2.6. Adaptación ligera de YOLOE con datos sintéticos — EN CURSO	6
2.7. Filtros de calidad — COMPLETADOS	14
2.8. Integración HSSD — COMPLETADA	15
2.9. Colocación de objetos personalizados — COMPLETADA	15
2.10. Bugs conocidos pendientes	15
2.11. Resumen de estado	16
3. Actualización actual: búsqueda de objetos en HSSD (15 mayo 2026)	16
3.1. Cambio de arquitectura: búsqueda <i>room-first</i>	16
3.2. Máquina de estados operativa	17
3.3. Reglas de confirmación YOLOE	17
3.4. Generación de puntos de exploración	17
3.5. Aproximación segura al objeto	18
3.6. Colocación de objetos personalizados	18
4. Los 5 métodos de búsqueda	18
4.1. M1: Búsqueda aleatoria (baseline)	18
4.2. M2: Candidato más cercano por BFS (implementado)	18
4.3. M3: Búsqueda semántica (VLMap quality ranking)	18
4.4. M4: Selección híbrida + navegación activa	19
4.5. M5: Sistema completo	19
5. Plan de implementación: próximos pasos	19

5.1. Paso A — Bugs inmediatos (prioridad alta, 1 sesión)	19
5.2. Paso B — Heatmap region analysis completo (1–2 sesiones)	19
5.3. Paso C — Baselines M1 y M3 (1 sesión)	20
5.4. Paso D — Razonamiento semántico iterativo con LLM (1–2 sesiones)	20
5.5. Paso E — Evaluación cuantitativa (1–2 sesiones)	20
5.6. Paso F — M4: fórmula híbrida + look-while-move (2–3 sesiones)	21
5.7. Paso G — M5: viewpoints + frontier exploration (3–4 sesiones)	21
5.8. Paso H — Wrapper ROS 1 Noetic para Toyota HSR (extensión)	21
5.9. Orden de implementación	21
6. Referencia: MoMa-LLM	21
7. Archivos clave del proyecto	22
8. Historial de actualizaciones	22
9. Situación actual y próximos pasos — 13 de abril de 2026	23
9.1. Estado del proyecto	23
9.2. Flujo completo del heatmap (implementado)	23
9.3. Resultados recientes — 24 de abril de 2026	25
9.4. Próximos pasos	26
10. Diario de desarrollo	28

1. Visión general del proyecto

1.1. Problema

Dado un objetivo expresado en lenguaje natural (por ejemplo, *"find the laptop"*), un robot simulado debe localizar el objeto de forma eficiente en un entorno interior. El robot opera en escenas Matterport3D y HSSD renderizadas con Habitat-Sim.

1.2. Hipótesis

Un sistema que combine tres niveles de conocimiento — etiquetas de habitaciones, memoria semántica de mobiliario (VLMaps, 37 categorías MP3D / 36 categorías HSSD) y verificación visual de vocabulario abierto (YOLOE) — encuentra objetos específicos de forma más eficiente que métodos que dependen de un solo nivel.

1.3. Los tres niveles de conocimiento

Nivel	Herramienta	Qué sabe
Habitación	LabelMe (MP3D) / <code>semantic_config.json</code> (HSSD)	«Esto es el salón»
Mueble / zona	VLMaps (37 cats MP3D / 36 cats HSSD)	«Hay un sofá allí»
Objeto específico	YOLOE (open-vocab)	«Hay un portátil encima de esa mesa»

1.4. Idea clave: VLMaps estrecha la búsqueda, YOLOE confirma

1. El usuario pide un objeto específico («laptop»).
2. Un LLM traduce ese objeto a categorías VLMap donde es probable encontrarlo: `laptop` → `[table, counter, shelving]`.
3. VLMaps localiza todas las instancias de esas categorías en el mapa.
4. El robot navega al candidato más prometedor.
5. YOLOE mira con la cámara y confirma si el objeto está ahí.
6. Si no → siguiente candidato. Si sí → **ENCONTRADO**.

1.5. Etiquetado de habitaciones

Para instrucciones del tipo *"go to the living room and find the laptop"*, el robot necesita saber dónde está el salón.

- **MP3D**: etiquetado manual offline con LabelMe (pipeline lista, pendiente de ejecutar).
- **HSSD**: anotaciones nativas en `semantics/scenes/<id>.semantic_config.json`. La clase `SemanticSceneRoomProvider` las rasteriza sobre el grid del VLMap sin necesidad de LabelMe.

2. Estado actual del proyecto (15 mayo 2026)

2.1. Infraestructura — **COMPLETADA**

- ✓ Docker con CUDA 12.1 + Habitat-Sim 0.3.1 + conda (Python 3.9)
- ✓ 3 escenas MP3D con VLMaps contruidos: `gTV8FGcVJC9_1`, `jh4fc5c5qoQ_1`, `mJXqzFtmKg4_1`

- ✓ 4 escenas adicionales descargadas (sin VLMa aún): mJXqzFtmKg4, jh4fc5c5qoQ, Je-FG25nYj2p, Uxmj2M2itWa
- ✓ 1 escena HSSD integrada: **102344280** (15 habitaciones, $\sim 447 \text{ m}^2$), VLMa construido e indexado
- ✓ Menú interactivo en Docker (`menu.sh`) para toda la pipeline
- ✓ Submodule git `third_party/vlmaps` (rama `tfg-modifications`)
- ✓ Repositorio limpio en GitHub: `mobileclip_blt.ts` (571 MB) eliminado del historial con `git-filter-repo`; push operativo

2.2. Navegación interactiva — COMPLETADA

- ✓ Pipeline completa: instrucción $\rightarrow$ LLM $\rightarrow$ goal selection $\rightarrow$ visgraph $\rightarrow$ navegación live con visualización
- ✓ `plan_path_only()`: planificación sin ejecución — elimina el teletransporte; camino mostrado como línea roja 800 ms antes de ejecutar
- ✓ Movimientos lentos y fluidos (`_NAV_STEP_DELAY_MS = 80 ms`)
- ✓ Dirección de cara corregida: $\theta = \arctan 2(-\Delta y, -\Delta x)$, signo de giro consistente con el planificador interno
- ✓ Goal selection con 3 niveles de fallback (room centroid / BFS / heatmap peak)
- ✓ Visgraph con margen de 4 px ($\sim 20 \text{ cm}$); recuperación de colisión (giro 30° + avance + replanificación); hasta 3 replanes y 8 candidatos
- ✓ Heatmap semántico overlay (JET colormap, 50 % opacidad)
- ✓ Cámara primera persona con inclinación -30°
- ✓ Multi-goal retry (hasta 8 candidatos por categoría, ordenados por BFS)

2.3. Mapper LLM — COMPLETADO

- ✓ El LLM traduce objetos no-VLMa a categorías VLMa probables. Ejemplo: `laptop` $\rightarrow$ `[table, counter, shelving]`
- ✓ Distingue queries directas (`sofa  $\in$  MP3D`) e indirectas (`laptop  $\notin$  MP3D`)
- ✓ Room-based reasoning: sugiere habitaciones probables además de muebles. Ejemplo: `laptop` $\rightarrow$ `rooms: [office, bedroom]`
- ✓ Prompt con 6 few-shot examples incluyendo rooms

2.4. Siguiente extensión semántica antes de ROS — objetos no-VLMa abiertos

Decisión táctica (24 abril 2026). Antes de migrar el proyecto a ROS 1 Noetic conviene estabilizar una capa semántica más rica para **objetos específicos que ya aparecen en HSSD pero no forman parte del vocabulario base de VLMas**. Esto permite validar la parte difícil del problema *semántico-visual* antes de mezclarla con problemas de ROS, Gazebo, `tf`, `move_base` o el HSR.

Tesis de diseño. El mapeo `objeto específico` $\rightarrow$ `muebles surrogate` no debe quedarse en una tabla manual cerrada. Para objetos como `bottle`, `charger`, `notebook` o `printer`, el sistema debe ser capaz de inferir dónde buscar sin tener que añadir una regla nueva cada vez. Por eso:

- el **LLM** debe decidir el *target canónico*, una lista ordenada de muebles surrogate y, opcionalmente, habitaciones probables;
- una **capa de validación determinista** debe filtrar esa salida y quedarse solo con categorías VLMaP realmente navegables;
- `object_priors.py` debe mantenerse como **fallback auditado**, no como única fuente de decisión.

Shortlist aprobada para la siguiente sesión.

- **Prioridad alta:** laptop, book, soap, vase, candle.
- **Prioridad media:** basket, printer, teapot.

Flujo objetivo. Para targets fuera del vocabulario VLMaP:

1. **Normalización léxica:** resolver alias triviales (`computer`→`laptop`, `books`→`book`, `teapot`→`teapot`, `soap dispenser`→`soap`).
2. **Consulta al LLM:** pedir una salida estructurada con `canonical_target`, `query_mode`, `surrogate_categories`, `likely_rooms`, `yoloe_label` y alias.
3. **Validación:** filtrar los surrogates contra `get_categories("hssd")` y contra el vocabulario realmente disponible en la escena.
4. **Planificación:** construir el heatmap con los muebles surrogate supervivientes.
5. **Verificación:** YOLOE confirma el `canonical_target`, no el surrogate. Es decir, se navega a una `table` o a un `desk`, pero se verifica `laptop`.

Objetivo práctico. La salida del LLM debe responder a *qué objeto es y sobre qué tipo de mueble conviene buscarlo*; el sistema debe seguir controlando *qué parte de esa sugerencia es ejecutable*. Esto deja el camino preparado para búsquedas más abiertas de nivel 3 y para una posterior migración a ROS con una semántica ya endurecida.

2.5. YOLOE como verificador — **COMPLETADO**

- ✓ Worker persistente (`_yoloe_persistent_worker.py`): modelo cargado una sola vez; inferencias ~100–500 ms en lugar de ~30 s
- ✓ Protocolo stdout correcto: diagnósticos a `sys.stderr`, `start()` lee hasta recibir `READY` (no solo una línea)
- ✓ `mobileclip_blt.ts` copiado automáticamente al arrancar (busca en `parents[2]`, `parents[4]`, `/workspace`; elimina versión corrupta de menos de 100 MB para forzar re-descarga)
- ✓ Captura de stderr del worker para diagnóstico de crashes
- ✓ Centrado visual iterativo (visual servoing): hasta 6 iteraciones de 5° ($\pm 15\%$ tolerancia)
- ✓ Flujo de verificación en 3 stages:
 1. Standoff 0,25 m — YOLOE síncrono
 2. Scan 360° in situ — YOLOE asíncrono (hilo bg)
 3. Componente alternativa del heatmap — YOLOE síncrono
- ✓ 15 categorías YOLOE-incompatibles saltan la verificación (`wall`, `floor`, `ceiling`, `beam`, `railing`, etc.)

✓ Caché global `get_session(target)`: reutiliza sesión entre instrucciones del mismo objeto

2.6. Adaptación ligera de YOLOE con datos sintéticos — EN CURSO

Decisión de alcance (junio 2026). La migración ROS/Gazebo/HSR queda como una integración funcional preliminar y trabajo futuro de ingeniería. La línea principal del TFG se refuerza en `tfg-sim`: mejorar el verificador visual que cierra la búsqueda semántica. Esto es coherente con la arquitectura: VLMaps propone habitaciones, muebles y superficies probables, mientras que YOLOE decide si el objeto pequeño está realmente visible desde la cámara.

Motivación. En las pruebas con objetos pequeños, el sistema ya localiza zonas razonables con VLMaps, pero a veces YOLOE detecta el objeto con confianza baja o lo pierde al acercarse. Esto afecta a objetivos como `mug`, `bottle`, `laptop`, `plush toy` y `ball`. Por tanto, la mejora más directa no es cambiar la memoria semántica, sino adaptar el verificador visual a los objetos domésticos usados en la evaluación.

Clases objetivo. Se ha fijado un conjunto pequeño y controlado de cinco clases: `mug`, `bottle`, `plush toy`, `laptop` y `ball`. Los modelos proceden de HSSD y se han seleccionado visualmente con mosaicos de candidatos. El objetivo no es entrenar un detector general, sino adaptar YOLOE al vocabulario de objetos de la evaluación end-to-end.

Captura de fondos. Se ha añadido un modo de captura manual de fondos RGB en Habitat-Sim, separado de la captura usada para construir VLMaps. El piloto se hizo en la escena HSSD 104862501_172226556, que no forma parte de las tres escenas principales usadas en las evaluaciones previas. El capturador guarda una imagen solo si el robot se ha desplazado al menos 1 m o ha cambiado la orientación al menos 30 grados, reduciendo fondos casi idénticos. Para la primera prueba se capturaron 100 fondos.

Generación sintética. El nuevo generador compone objetos HSSD sobre los fondos capturados. Cada objeto se renderiza con Habitat-Sim contra un fondo limpio usando sensor RGB y sensor semántico. La máscara semántica da un polígono de segmentación en formato YOLO, sin etiquetado manual. La caja 2D se mantiene solo para revisión visual y metadatos. Después se aplica composición aleatoria con restricciones por clase: escalas, orientación, traslación, negativos y hasta tres objetos por imagen. La pelota puede rotar libremente; en objetos con orientación clara se evita generar vistas físicamente absurdas.

Piloto generado. Con los 100 fondos se generó un piloto de 200 imágenes en `/shared/yoloe_synthetic/piloto`. La división es 70/20/10 y las etiquetas son polígonos de segmentación con unas 26–28 coordenadas de media por instancia:

- **train:** 140 imágenes, 223 instancias y 19 negativos.
- **validation:** 40 imágenes, 63 instancias y 9 negativos.
- **test:** 20 imágenes, 29 instancias y 5 negativos.

Selección entre modelo base y modelo adaptado. Se ha añadido una ruta explícita para comparar ambos detectores sin tocar la lógica semántica:

- `VLMAPS_YOLOE_WEIGHTS`: variable de entorno para navegación interactiva. Si está vacía se usa el modelo base.
- `-yoloe-weights`: argumento en `run_nav_eval.py`, `run_full_eval.py`, `run_open_vocab_eval.py`, `run_yoloe_thresh_sweep.py` y `run_orchestrator_phase_eval.py`.

mug candidates - USED = anchor from useful scenes

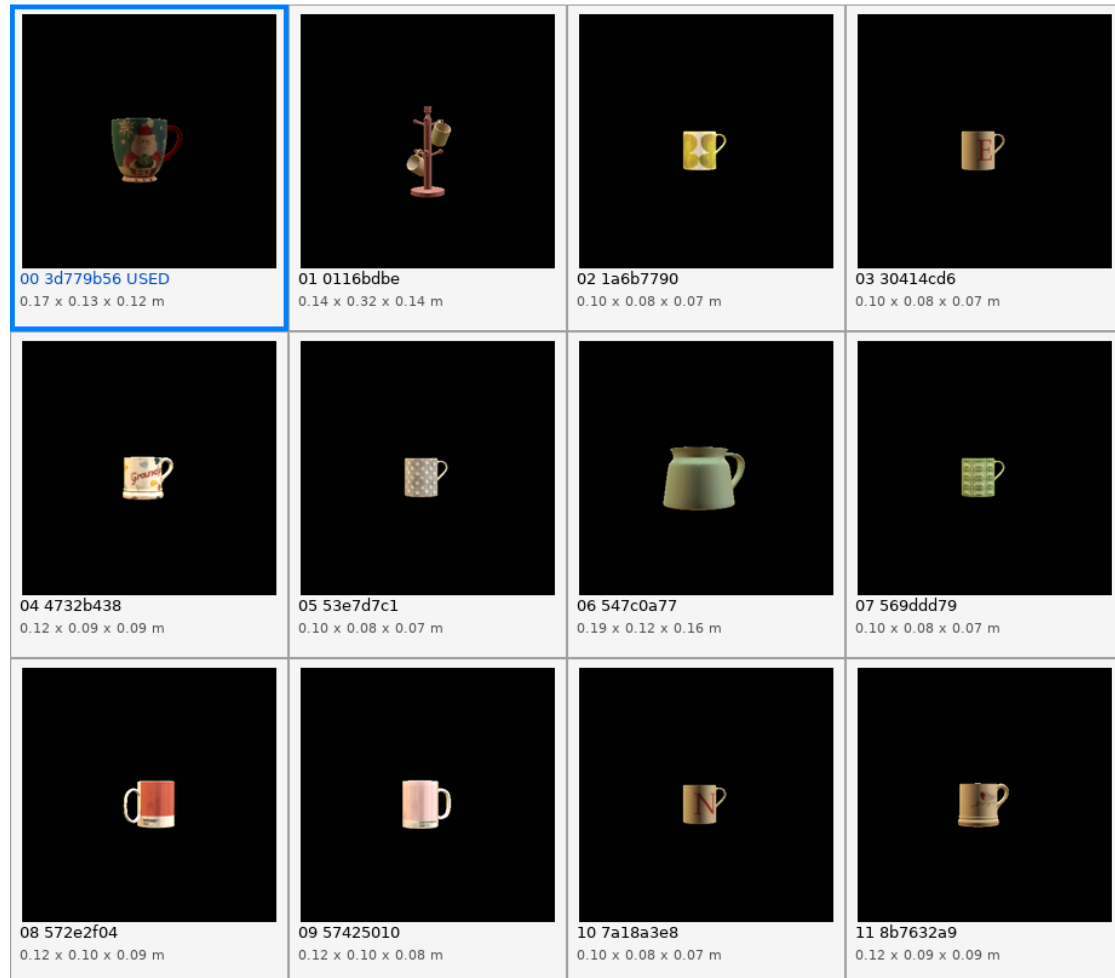


Figura 1: Mosaico de candidatos HSSD para mug. Se eligieron seis variantes visuales para cubrir cambios de forma, color y asa.

bottle candidates - USED = anchor from useful scenes

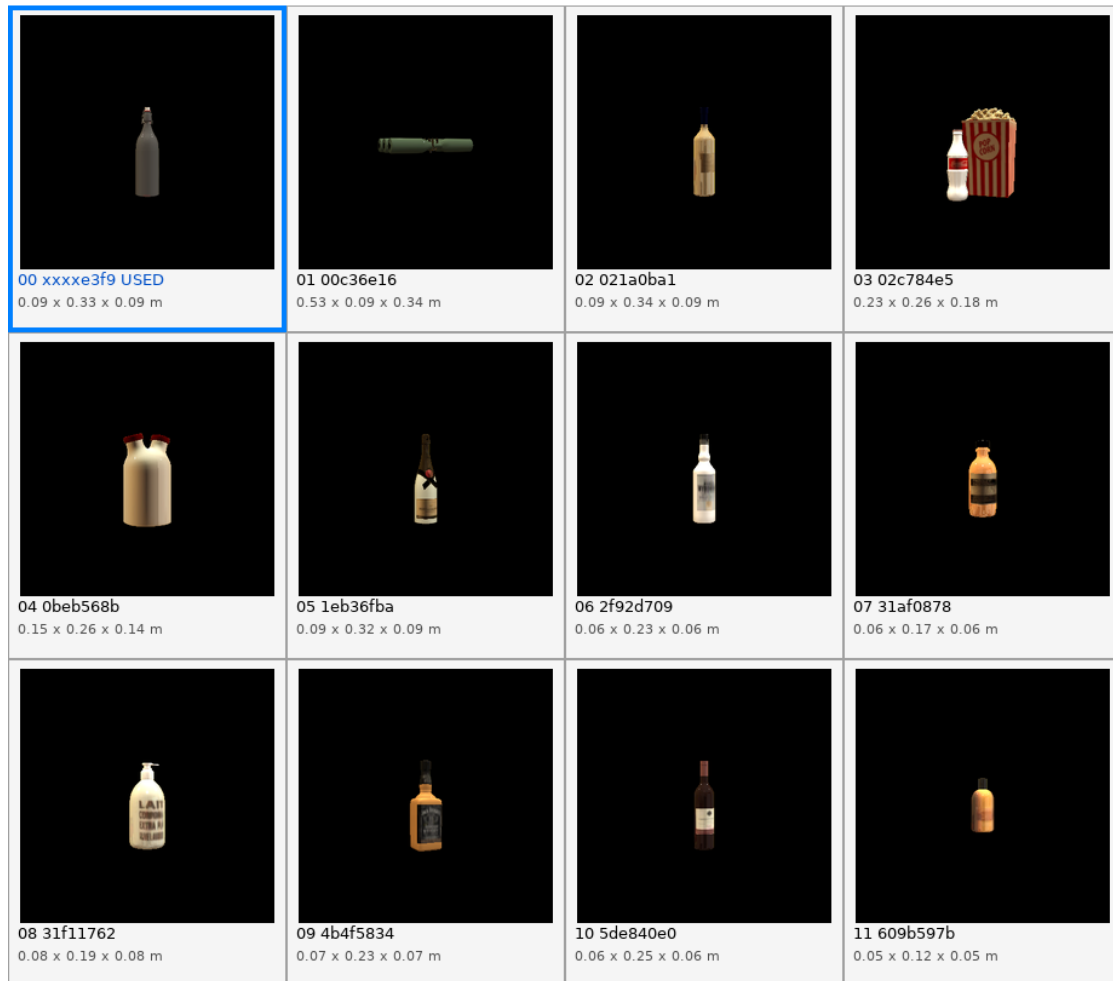


Figura 2: Mosaico de candidatos HSSD para bottle. Se eligieron cuatro modelos con siluetas y materiales distintos.

plush toy candidates - USED = anchor from useful scenes

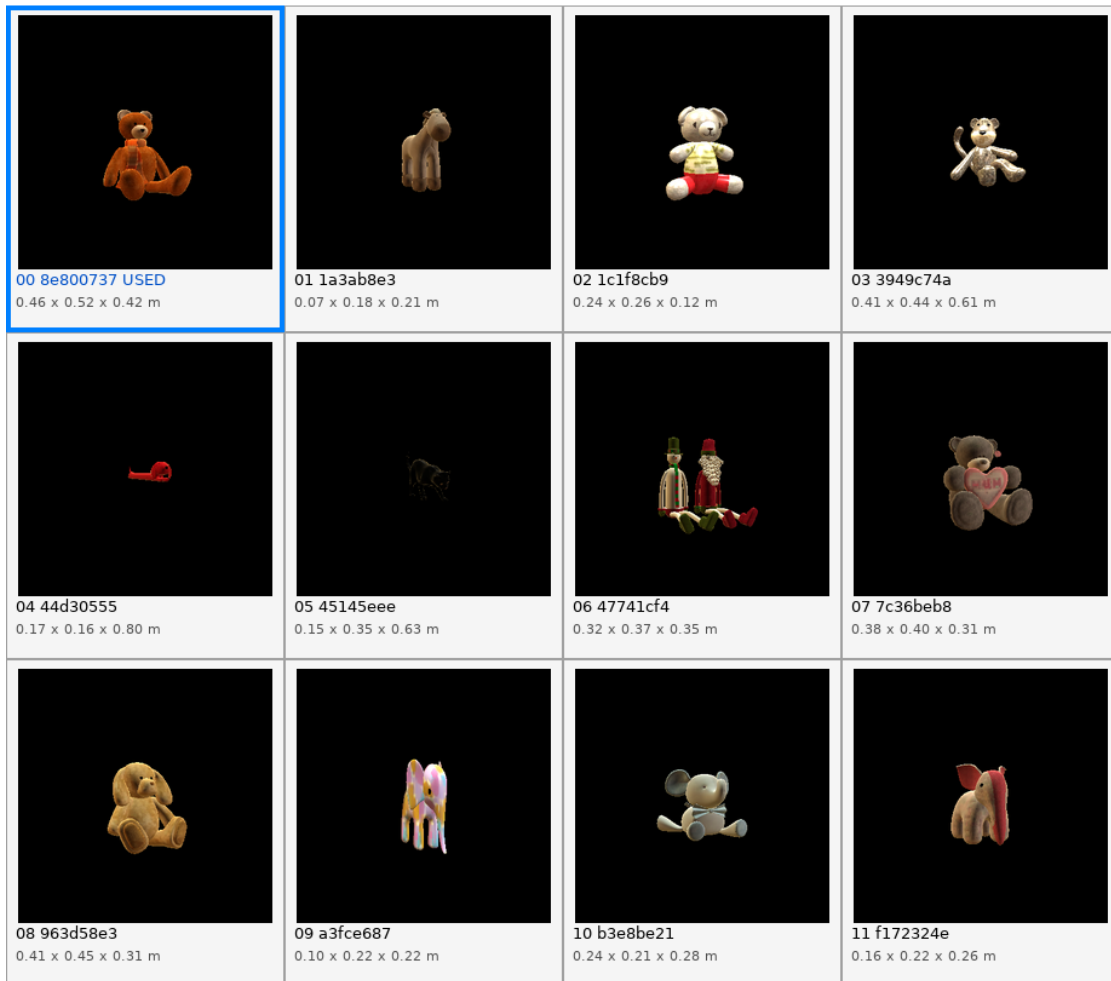


Figura 3: Mosaico de candidatos HSSD para plush toy. La clase se define como peluche genérico, no necesariamente oso.

laptop candidates - USED = anchor from useful scenes

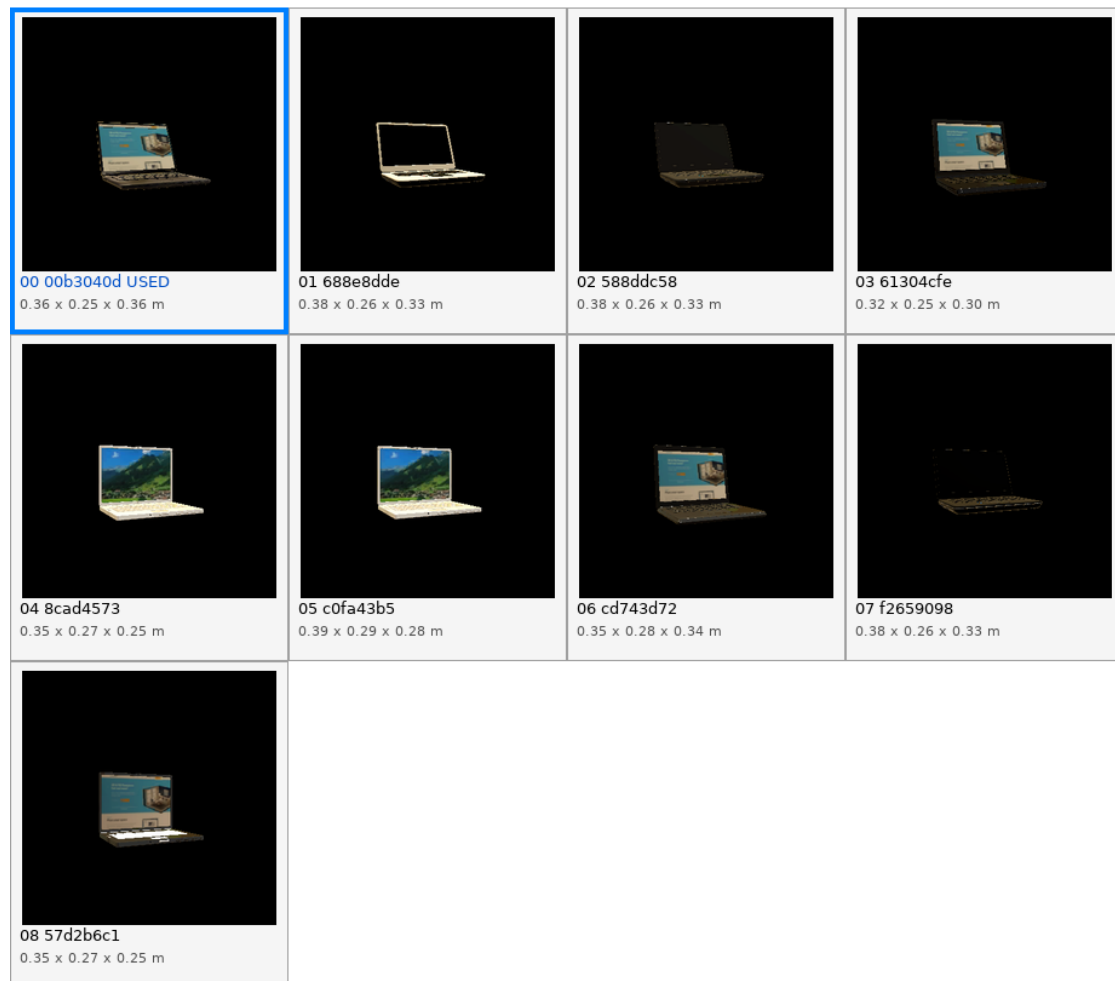


Figura 4: Mosaico de candidatos HSSD para laptop. Se conservan tres variantes porque la clase es visualmente más estable.

basketball candidates - USED = anchor from useful scenes

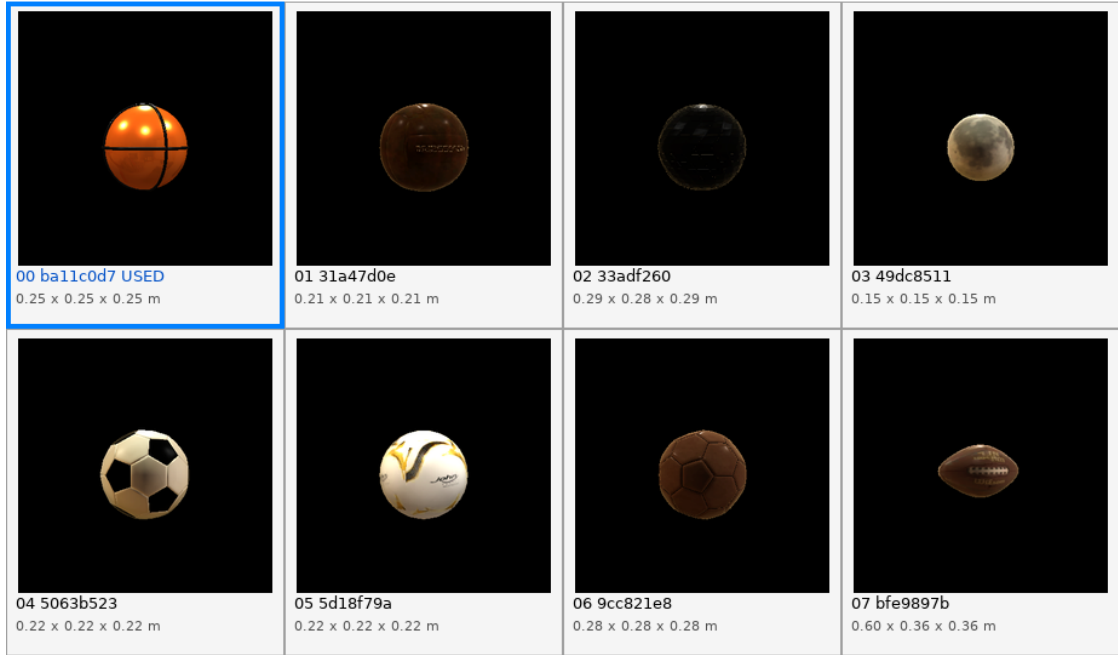


Figura 5: Mosaico de candidatos HSSD para ball. Se usan tres variantes para cubrir balones con apariencia diferente.



Figura 6: Muestra de fondos interiores capturados manualmente para el dataset sintético. No se usan para construir VLMaps, solo como contexto visual para el detector.

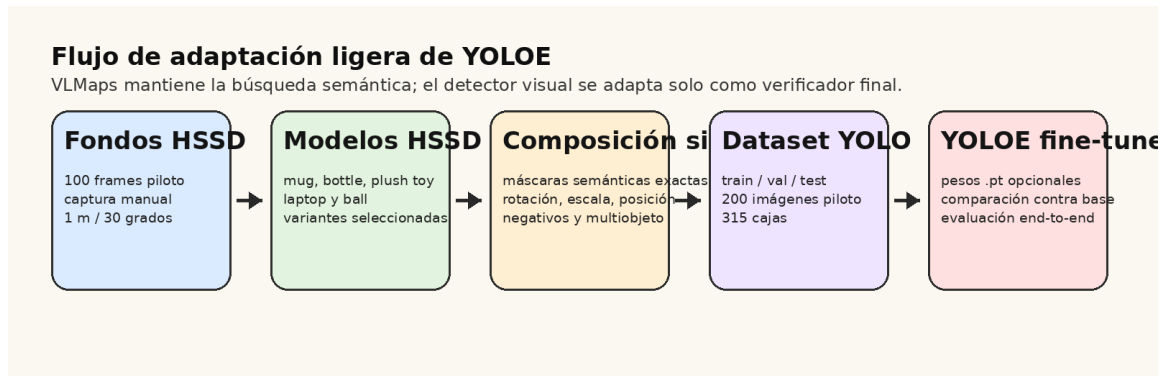


Figura 7: Flujo de adaptación ligera: captura de fondos, render de modelos HSSD, composición sintética, dataset YOLO y comparación YOLOE base frente a YOLOE adaptado.

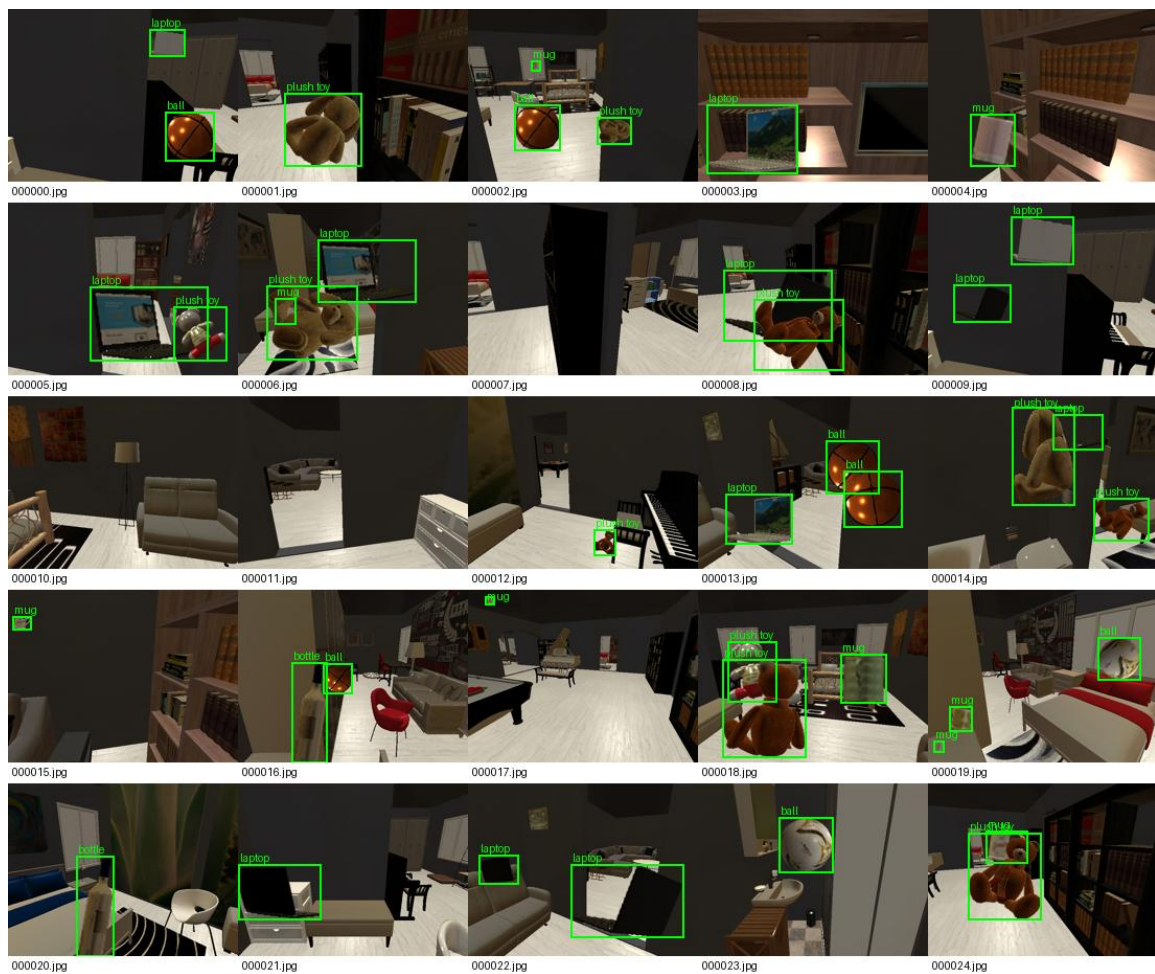


Figura 8: Mosaico de revisión del dataset sintético piloto. Las cajas mostradas son una ayuda visual; el entrenamiento usa polígonos derivados de máscaras semánticas.

- El menú Docker pregunta por la ruta `.pt` en evaluaciones y navegación manual. En blanco significa YOLOE base.
- El menú también incluye una opción de entrenamiento sobre datasets sintéticos, con salida en `/shared/yoloe_finetune`. El archivo `weights/best.pt` resultante es la ruta que se introduce después para comparar contra el modelo base.
- Los manifiestos de evaluación registran `yoloe_weights` para que las tablas de resultados distingan claramente el modelo usado.

Smoke test de entrenamiento. Se ejecutó una prueba mínima de una época sobre el piloto segmentado, con `imgsz=320` y `batch=1`, en la GPU RTX 3070 Ti Laptop. Esta prueba no se usa como resultado final, pero valida que el formato de labels es aceptado por `YOLOESegModel`, que no hay imágenes corruptas y que se produce un `best.pt` reutilizable: `/shared/yoloe_finetune/smoke_pilot_200_`. En validación, tras una sola época, se obtuvo `mAP50(B)=0.521`, `mAP50-95(B)=0.445`, `mAP50(M)=0.517` y `mAP50-95(M)=0.376`. Estos valores no deben interpretarse como conclusión experimental, sino como comprobación de viabilidad del flujo.

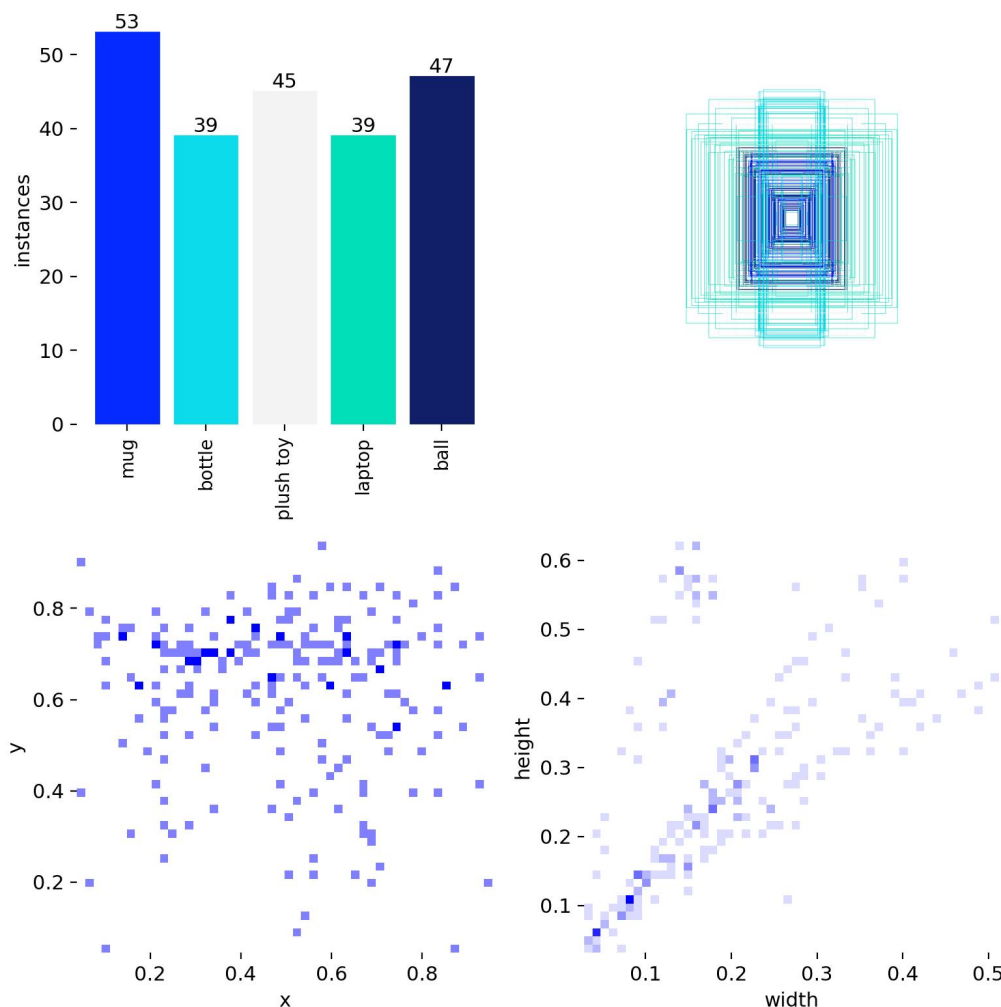


Figura 9: Distribución visual de etiquetas generada por Ultralytics durante el smoke test de fine-tuning.

Primer entrenamiento piloto. Se entrenó el dataset `pilot_200_manual_100bg` durante 30 épocas a `imgsz=640` y `batch=2`. El entrenamiento paró por early stopping en la época 28 y el mejor

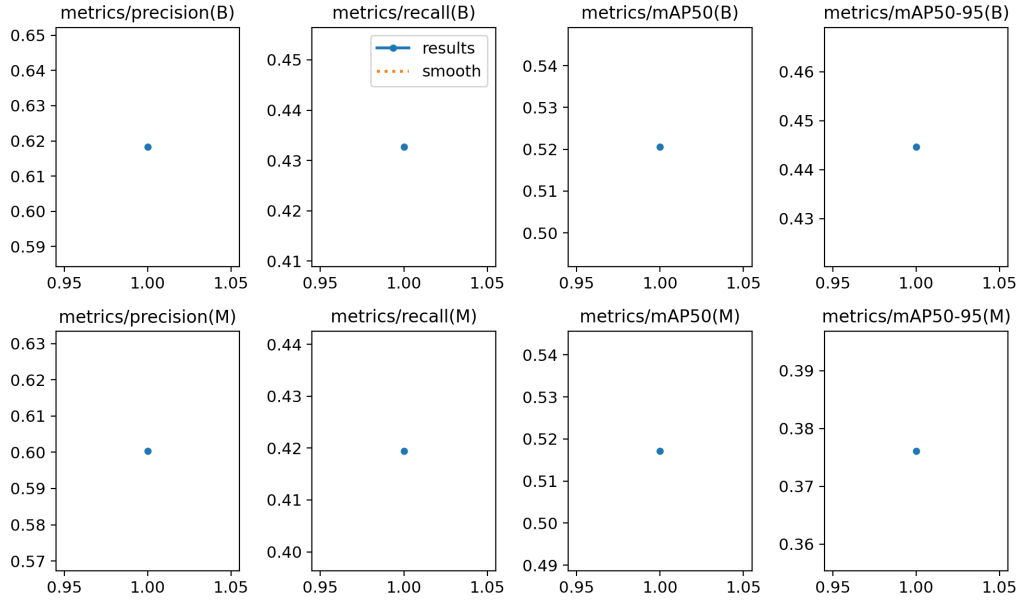


Figura 10: Curvas de entrenamiento y validación del smoke test de una época. Su función es verificar el pipeline, no cerrar la evaluación final.

checkpoint se obtuvo en la época 18: `/shared/yoloe_finetune/pilot_200_e30/weights/best.pt`. En validación interna, el modelo alcanzó $\text{mAP50(B)}=0.720$, $\text{mAP50-95(B)}=0.670$, $\text{mAP50(M)}=0.736$ y $\text{mAP50-95(M)}=0.647$. Sin embargo, la comparación en el split `test` muestra que todavía no debe sustituir al YOLOE base en navegación:

Modelo	P(M)	R(M)	mAP50(M)	mAP50-95(M)
YOLOE base	0.761	0.614	0.702	0.550
YOLOE adaptado piloto	0.824	0.392	0.643	0.551

El piloto mejora la precisión, pero reduce claramente el recall. Para búsqueda semántica el criterio operativo principal es no perder objetos visibles, por lo que este checkpoint queda como prueba de viabilidad del pipeline, no como modelo recomendado para las evaluaciones end-to-end. La causa probable es que 200 imágenes son pocas para adaptar el detector sin sobreajuste y el split de test contiene muy pocas instancias por clase.

Siguiente experimento. El siguiente paso es entrenar un primer modelo real con más datos, no más épocas sobre el piloto. La siguiente versión debe generar al menos 1500 imágenes sintéticas, aumentar la variedad de posiciones y reforzar las clases con peor recall, especialmente `mug` y `laptop`. Si el nuevo modelo mejora recall sin disparar falsos positivos, entonces se usa en las evaluaciones end-to-end mediante `-yoloe-weights`; si no, se conserva YOLOE base y se documenta el fine-tuning como análisis experimental acotado.

2.7. Filtros de calidad — COMPLETADOS

- ✓ **Filtro de coherencia de heatmaps:** mín. 30 celdas, $\geq 30\%$ dominancia, ≤ 20 componentes conexas
- ✓ **Filtro por tamaño medio de regiones:** descarta contornos $< 30\%$ del área media
- ✓ **Wall filter:** ratio BFS/euclídea $> 3\times$ descarta candidatos al otro lado de paredes
- ✓ **Postprocess heatmap:** suavizado Gaussiano $\rightarrow$ umbral relativo $\rightarrow$ metadatos por com-

ponente (área, score, densidad, centroide)

2.8. Integración HSSD — **COMPLETADA**

- ✓ `dataset_type=hssd` activa la rama HSSD sin romper MP3D
- ✓ `scene_dataset_config_file` declarado en Hydra config (sin prefijo +), corregido en `menu.sh`
- ✓ Navmesh recomputado al iniciar con parámetros correctos
- ✓ 36 categorías HSSD en `get_categories("hssd")`
- ✓ `SemanticSceneRoomProvider`: lee `semantic_config.json` de HSSD y rasteriza polígonos de habitación sobre el grid VLMap
- ✓ Navegación interactiva validada en HSSD escena 102344280

2.9. Colocación de objetos personalizados — **COMPLETADA**

- ✓ `place_objects.py`: define objetos 3D (.glb) via JSON config
- ✓ Carga automática al iniciar; objetos visibles para YOLOE pero NO en VLMap

2.10. Bugs conocidos pendientes

- **IndexError boundary**: goal exactamente en el borde del obstacle map (`index 193 is out of bounds for size 193`). Fix: clamp en `plan_path_only()` antes de `plan_to()`
- **YOLOE conf_thresh bajo**: umbral actual 0.25; demasiado permisivo para verificación de llegada

2.11. Resumen de estado

Componente	Estado
Infraestructura Docker + Habitat + VLMaps	Hecho
Navegación interactiva M2 (BFS + colisiones)	Hecho
Mapper LLM (objeto $\rightarrow$ cats + rooms)	Hecho
YOLOE persistente + scan 360°	Hecho
Visual centering (servo YOLOE)	Hecho
Filtros de calidad (coherencia, área, pared)	Hecho
Integración HSSD completa	Hecho
Camino sin teletransporte	Hecho
Room-based reasoning (LLM)	Hecho
Colocación objetos personalizados	Hecho
Bug: IndexError boundary	Pendiente fix
Bug: conf_thresh YOLOE bajo	Pendiente fix
Heatmap region analysis (fórmula quality)	Pendiente
M1 baseline (aleatorio)	Pendiente
M3 baseline (VLMap quality ranking)	Pendiente
LLM iterativo con contexto de búsqueda	Pendiente
Extensión semántica abierta (objetos no-VLMap)	Siguiente sesión
M4: fórmula híbrida + look-while-move	Pendiente
M5: viewpoints + frontier exploration	Pendiente
Evaluación cuantitativa automática	Pendiente
Wrapper ROS 1 Noetic (Toyota HSR)	Pendiente
Etiquetado LabelMe (MP3D, 3 escenas)	Pipeline lista
VLMaps JeFG25nYj2p, Uxmj2M2itWa	Escenas descargadas

3. Actualización actual: búsqueda de objetos en HSSD (15 mayo 2026)

3.1. Cambio de arquitectura: búsqueda *room-first*

El flujo actual ya no debe empezar inspeccionando candidatos sueltos del heatmap global. Para objetos pequeños o de vocabulario abierto, la política correcta es:

1. Parsear la instrucción y resolver el objeto canónico para YOLOE.
2. Elegir primero la habitación: habitación explícita del usuario si existe; si no, habitación fijada por priors/LLM.
3. Navegar a esa habitación usando exclusivamente la región LabelMe/manual disponible en `room_map`.
4. Generar una sola vez los puntos de exploración dentro de la zona navegable de esa habitación, enfocados por muebles o zonas VLMaps candidatas (`counter`, `table`, `cabinet`, etc.) cuando haya evidencia útil.
5. Ejecutar una única exploración de habitación. No hay una rutina previa de *pin-sweep* ni visitas a muebles antes de generar los puntos.

Esto unifica dos casos que antes se comportaban distinto: “*find the bottle*” y “*go to the kitchen and find the bottle*”. Ambos deben entrar primero en la habitación objetivo y después ejecutar la misma máquina de estados de búsqueda.

3.2. Máquina de estados operativa

Estado	Responsabilidad
PARSE_INSTRUCTION	Resolver objetivo canónico, etiqueta YOLOE, muebles surrogate y habitaciones probables.
ROOM_SELECT	Elegir habitación: explícita/pinned > priors > likely rooms del LLM.
ROOM_STAGE	Mover el robot hasta una celda segura dentro de la habitación LabelMe/manual.
GENERATE_EXPLORE_POINTS	Construir puntos de exploración una sola vez sobre espacio navegable real de la habitación.
ZONE_EXPLORE	Recorrer el circuito de puntos. En cada punto se hace barrido limitado; durante el trayecto YOLOE puede disparar una detección de baja confianza.
APPROACH_DETECTION	Si YOLOE detecta con umbral bajo ($\geq 0,40$), se aborta la exploración y se pasa a aproximación. La detección baja nunca cuenta como éxito.
NAVIGABLE_APPROACH	Calcular el punto navegable libre más cercano a la detección y aproximarse con $A^*/navmesh$, manteniendo YOLOE activo durante el movimiento. El avance frontal directo queda solo como fallback corto.
FINAL_CONFIRMATION	Confirmar desde cerca. Si YOLOE $\geq 0,80$, el objetivo queda encontrado. Si YOLOE $\geq 0,75$ pero $< 0,80$, se marca como alta probabilidad y se detiene la exploración.
FOUND / HIGH_PROBABILITY / FAILED	Estados terminales de la instrucción.

3.3. Reglas de confirmación YOLOE

- **Baja confianza** ($\geq 0,40$): solo dispara aproximación; nunca es éxito.
- **Alta probabilidad** ($\geq 0,75$): se considera que el objeto probablemente está visto; se aborta la exploración para no seguir buscando otro objeto parecido.
- **Confirmación final** ($\geq 0,80$): éxito real. Se congela la detección visual y el mapa marca el objetivo como encontrado.
- Durante NAVIGABLE_APPROACH, YOLOE se evalúa en cada paso. Si aparece una confirmación $\geq 0,80$, se para la ruta inmediatamente y se marca éxito.

3.4. Generación de puntos de exploración

La generación actual se basa en la máscara navegable de la habitación, no en una caja rectangular. El selector usa:

- `room_mask & free_map` como dominio real.
- Candidatos por niveles de clearance para no perder zonas estrechas.
- Agrupación/cobertura espacial sobre celdas navegables reales.
- Ordenación como circuito empezando por el punto más cercano al robot.
- Depuración visual en `data/vlmaps_dataset_hssd/<scene>/debug/`.

El ratio automático se ha reducido para evitar demasiados puntos. El menú usa por defecto modo

automático con base menor y cap reducido; si una habitación es grande se añaden más puntos, pero no tantos como para convertir la búsqueda en exploración exhaustiva.

3.5. Aproximación segura al objeto

El comportamiento deseado no es “ir recto hacia la botella”. Cuando YOLOE detecta un objeto, la aproximación debe:

1. estimar la celda objetivo desde la vista actual;
2. buscar un standoff navegable alrededor de esa celda;
3. planificar con A*/navmesh respetando obstáculos y, si aplica, la habitación fijada;
4. mantener YOLOE activo durante la ruta;
5. confirmar en cuanto la confianza supere 0,80.

Se ha eliminado la orientación hacia “evidencia VLMaps cercana” al llegar a un punto de exploración. Esa acción confundía el flujo: hacía que el robot girase hacia muebles candidatos en vez de mantener el objetivo visual detectado.

3.6. Colocación de objetos personalizados

Los objetos añadidos manualmente en HSSD se regeneran con separación mínima entre instancias para evitar solapamientos. En la escena 108736884_177263634_4, los objetos de cocina quedan separados en el plano XZ por más de 2 m entre sí, evitando el problema anterior de objetos superpuestos sobre el mismo counter/cabinet. También se retiraron los libros rosas añadidos para pruebas anteriores.

4. Los 5 métodos de búsqueda

Todos comparten la misma infraestructura (visgraph, navegación, visualización). Lo que cambia es la **estrategia de selección de goal** y el comportamiento durante el tránsito.

4.1. M1: Búsqueda aleatoria (baseline)

- **Goal selection:** punto aleatorio navegable del mapa.
- **Verificación:** YOLOE al llegar.
- **Si no encuentra:** otro punto aleatorio.
- **Estado:** No implementado.

4.2. M2: Candidato más cercano por BFS (implementado)

- **Goal selection:** si el objeto está en VLMap, BFS al más cercano. Si no, mapper LLM → categorías relacionadas → BFS al mueble más cercano.
- **Verificación:** YOLOE al llegar (scan 0° , $\pm 30^\circ$) para indirectos. VLMap para directos.
- **Si no encuentra:** siguiente candidato por distancia (hasta 8 candidatos).
- **Estado:** Implementado y funcional.

4.3. M3: Búsqueda semántica (VLMap quality ranking)

- **Goal selection:** candidatos ordenados por heatmap region quality (fórmula: $0,3 \cdot \text{area} + 0,4 \cdot \text{mean_score} + 0,3 \cdot \text{density}$).

- **Verificación:** YOLOE al llegar.
- **Estado:** No implementado (requiere Paso B).

4.4. M4: Selección híbrida + navegación activa

- **Goal selection:** fórmula híbrida $\text{score}(c) = \alpha \cdot \text{visual}(c) + \beta \cdot \text{semantic}(c) - \gamma \cdot \text{cost}(c)$.
- **Durante tránsito:** look-while-move + re-routing si detecta candidato mejor.
- **Estado:** No implementado.

4.5. M5: Sistema completo

- **Goal selection:** igual que M4 + viewpoint planning (4 poses alrededor) + frontier exploration.
- **Estado:** No implementado.

Método	Goal sel.	Look-move	Re-route	Viewpoints	Estado
M1	Random	No	No	No	Pendiente
M2	BFS dist	No	No	No	Hecho
M3	VLMa quality	No	No	No	Pendiente
M4	$\alpha v + \beta s - \gamma c$	Sí	Sí	Frontal	Pendiente
M5	$\alpha v + \beta s - \gamma c$	Sí	Sí	4 ángulos	Pendiente

5. Plan de implementación: próximos pasos

5.1. Paso A — Bugs inmediatos (prioridad alta, 1 sesión)

A1 — IndexError boundary (1 línea). En `plan_path_only()` de `habitat_lang_robot.py`, clamp antes de llamar a `self.nav.plan_to()`:

```
pos = [
    min(pos[0], self.nav.obs_map.shape[0] - 1),
    min(pos[1], self.nav.obs_map.shape[1] - 1),
]
```

No se modifica `navigation_utils.py` (código original de `vlmaps`). El clamp va en el código propio, antes de la llamada.

A2 — Umbral de confianza YOLOE. Subir `conf_thresh`:

- Stage 1 y 3 (verificación de llegada): **0.45** — si el robot está frente al objeto, debe verlo con alta confianza.
- Stage 2 (scan 360°): **0.30** — el objeto puede aparecer parcialmente o de lado durante el giro.

5.2. Paso B — Heatmap region analysis completo (1–2 sesiones)

Implementar la función de calidad por región del heatmap:

$$\text{quality}(r) = 0,3 \cdot \text{norm}(\text{area}) + 0,4 \cdot \text{mean_score} + 0,3 \cdot \text{density}$$

Donde:

- `norm(area)`: área del componente normalizada respecto al mayor componente

- `mean_score`: puntuación CLIP media de los voxels del componente
- `density`: fracción de celdas activas dentro del bounding box del componente

Los metadatos por componente ya se calculan en `postprocess_heatmap()` (área, max, media, bbox, centroide, score). Solo hay que combinarlos en esta fórmula y exponerlos para que el selector de goal los use.

Esto es el prerequisite directo para M3.

5.3. Paso C — Baselines M1 y M3 (1 sesión)

M1 (aleatorio): seleccionar un punto navegable aleatorio del mapa como goal. Sirve como baseline mínimo para demostrar que VLMs aporta valor.

M3 (semántico): en lugar de ir al candidato más cercano por BFS (M2), ir al candidato con mayor `quality score` (Paso B). Esto mide si la información semántica del heatmap mejora sobre la proximidad pura.

5.4. Paso D — Razonamiento semántico iterativo con LLM (1–2 sesiones)

Inspirado conceptualmente en MoMa-LLM (Honerkamp et al., RSS SemRob 2024). Implementación propia desde cero, sin uso del código del repositorio de Toyota Motor Europe.

Mejorar el Mapper LLM para incluir **contexto del estado actual de búsqueda**. En lugar de que el LLM decida categorías una sola vez al principio, recibe también:

- Qué habitaciones tiene la escena (de `SemanticSceneRoomProvider` para HSSD o LabelMe para MP3D)
- Qué candidatos se han visitado ya y cuál fue el resultado (fallado / confirmado por YOLOE)
- Qué habitaciones se han explorado

El LLM recibe este contexto estructurado y decide dónde buscar a continuación. Ejemplo:

Scene rooms: kitchen, living_room, bedroom_1, bedroom_2

Already searched:

- table in living_room: YOLOE not found
- counter in kitchen: YOLOE not found

Remaining candidates: shelving in office, table in bedroom_1

Where should the robot go next, and why?

Esto convierte M2 en un método adaptativo. Se implementa como extensión del Mapper LLM existente, añadiendo el historial de búsqueda como campo adicional en el prompt.

5.5. Paso E — Evaluación cuantitativa (1–2 sesiones)

Script automático que ejecuta N trials con objetos conocidos y mide:

- **Success Rate (SR)**: fracción de episodios donde el objeto se encuentra
- **Correct First Goal**: fracción donde el primer candidato visitado es correcto
- **Path Length (SPL)**: distancia total recorrida normalizada por el óptimo
- **Wrong Visits**: candidatos visitados antes del correcto
- **Time to Find**: pasos de simulación hasta encontrar

Output: CSV con resultados + gráficas comparativas M1 vs M2 vs M3 para la memoria del TFG.

Para HSSD: usar los objetos con anotaciones nativas del `semantic_config.json` para generar trials con ground truth automatizado.

5.6. Paso F — M4: fórmula híbrida + look-while-move (2–3 sesiones)

Goal selection con fórmula híbrida:

$$\text{score}(c) = \alpha \cdot \text{visual}(c) + \beta \cdot \text{semantic}(c) - \gamma \cdot \text{cost}(c)$$

Durante el tránsito: si el robot pasa cerca de un hotspot VLMaap relevante, ejecuta una verificación YOLOE lateral sin detenerse. Si detecta el objeto, re-routing inmediato.

5.7. Paso G — M5: viewpoints + frontier exploration (3–4 sesiones)

Viewpoints: al llegar a un candidato, el robot se coloca en 4 puntos alrededor (a 1,5 m, cada 90°) y ejecuta YOLOE desde cada uno. Reduce falsos negativos por oclusión.

Frontiers: si se agotan todos los candidatos, explorar zonas inexploradas priorizando las cercanas a regiones VLMaap relevantes.

5.8. Paso H — Wrapper ROS 1 Noetic para Toyota HSR (extensión)

- `vlmap_server` node: carga VLMaap, expone servicio `/vlmap/query`
- `vlmap_nav` node: recibe instrucciones, parsea con LLM, envía goals a `move_base`
- Visualización en RViz: heatmap como `OccupancyGrid`, path como `nav_msgs/Path`

5.9. Orden de implementación

Prioridad	Paso	Depende de	Estado
—	M2 + YOLOE persistente	—	Hecho
—	HSSD integración completa	—	Hecho
1	A: Fix IndexError boundary + conf_thresh	—	Pendiente
2	B: Heatmap region analysis	—	Pendiente
3	C: M1 + M3 baselines	Paso B	Pendiente
4	D: LLM iterativo con contexto	—	Pendiente
5	E: Evaluación cuantitativa	Pasos C, D	Pendiente
6	F: M4 híbrido	Paso B	Pendiente
7	G: M5 viewpoints + frontiers	Paso F	Pendiente
8	H: Wrapper ROS Noetic	—	Pendiente
—	Etiquetado LabelMe (MP3D)	—	Pipeline lista
—	VLMaaps JeFG25nYj2p, Uxmj2M2itWa	—	Pendiente

6. Referencia: MoMa-LLM

El paper *Language-Grounded Dynamic Scene Graphs for Interactive Object Search with Mobile Manipulation* (Honerkamp et al., RSS SemRob 2024) propone construir un scene graph dinámico mientras el robot explora y usar un LLM para razonar sobre él.

El código asociado (<https://github.com/robot-learning-freiburg/MoMa-LLM>) es propiedad de Toyota Motor Europe NV/SA y no puede usarse sin licencia expresa. Ninguna línea de ese código se utiliza en este TFG.

Las ideas conceptuales del paper (representación estructurada del estado para el LLM, room classification a partir de objetos observados, scoring semántico de fronteras) se usan como inspiración para el Paso D, con implementación completamente propia.

Diferencia principal: MoMa-LLM construye el mapa dinámicamente durante la exploración; este TFG usa un VMap pre-construido, lo que permite evaluación más controlada y comparación directa entre métodos (M1–M5).

7. Archivos clave del proyecto

Archivo	Rol
application/interactive_object_nav.py	Pipeline de navegación principal
vlmaps/robot/habitat_lang_robot.py	Robot + visgraph + plan_path_only
vlmaps/utils/llm_utils.py	Mapper LLM: objeto $\rightarrow$ categorías + habitaciones
vlmaps/utils/yoloe_utils.py	YoloeSession: worker persistente, sync/async
vlmaps/utils/_yoloe_persistent_worker.py	Subprocess YOLOE aislado
vlmaps/utils/room_provider.py	SemanticSceneRoomProvider (HSSD nativo)
vlmaps/utils/navigation_utils.py	plan_to_pos_v2 : visgraph + BFS
config/object_goal_navigation_cfg.yaml	Config Hydra principal
docker/menu.sh	Menú interactivo Docker

8. Historial de actualizaciones

31 de marzo de 2026 — Correcciones del pipeline HSSD

1. **Mapa de obstáculos invertido**: modelo de dos bandas (suelo $h < 0,15$ m / obstáculo $0,2 \leq h < 1,5$ m). Una celda navegable = 1 si tiene suelo Y no tiene obstáculo.
2. **Heatmap ruidoso**: argmax binario reemplazado por score CLIP continuo con umbral 0.3 + max-pooling + difusión solo desde celdas con score alto.
3. **Robot atraviesa paredes en HSSD**: `sim.recompute_navmesh()` en `_setup_sim()` cuando `dataset_type == "hssd"`.
4. **Integración YOLOE**: `yoloe_utils.py` + `_yoloe_worker.py`, superposición sobre ventana 1st person.
5. **Camino visible**: `path_cells` pasado a `show_map()`, color rojo.
6. **Cámara primera persona**: inclinación $-\pi/8$ rad ($\approx 22,5^\circ$).

2 de abril de 2026 (primera iteración)

1. **Postproceso de heatmaps**: suavizado Gaussiano $\rightarrow$ umbral relativo $\rightarrow$ metadatos por componente (área, max, media, bbox, centroide).
2. **Exploración 360°**: pasos de 20° , detención al detectar.
3. **Ruta alternativa**: componente con mayor score a >20 celdas.
4. **Movimiento fluido**: sin `set_agent_state()` visible.

2 de abril de 2026 (segunda iteración)

1. **Worker YOLOE persistente**: modelo cargado una sola vez; protocolo stdin/stdout; inferencias $\sim 100\text{--}500$ ms.

2. **YoloeSession**: modos síncrono y asíncrono (hilo bg).
3. **Scan 360° fluido**: pasos nativos de 5°, YOLOE en hilo bg.
4. **Cámara -30°**: `habitat_utils.py`.
5. **Standoff 0,25 m**: acercamiento máximo al objeto.

13 de abril de 2026

1. **Bug protocolo stdout/stderr YOLOE**: `_ensure_mobileclip()` imprimía en stdout interceptando el `READY`; corregido a `sys.stderr.start()` ahora lee hasta recibir `READY` (no solo una línea).
2. **mobileclip_blt.ts**: búsqueda en múltiples rutas candidatas; eliminación automática de versión corrupta (menos de 100 MB).
3. **Captura stderr del worker**: crashes visibles en consola.
4. **Dirección de cara definitiva**: $\arctan 2(-\Delta y, -\Delta x)$, signo consistente con el planificador.
5. **Sin teletransporte**: `plan_path_only()` + camino 800 ms antes de ejecutar.
6. **Hydra corregido**: `scene_dataset_config_file` sin prefijo + en ejecución y documentación de `menu.sh`.
7. **Repositorio GitHub limpio**: `mobileclip_blt.ts` (571 MB) eliminado del historial con `git-filter-repo`; push operativo.
8. **Bugs conocidos**: `IndexError boundary` + `conf_thresh` YOLOE bajo.

9. Situación actual y próximos pasos — 13 de abril de 2026

9.1. Estado del proyecto

A fecha de 13 de abril de 2026, el sistema de navegación semántica M2 está completamente operativo en escenas MP3D y HSSD. El robot recibe una query en lenguaje natural, genera un heatmap semántico con VLMaps + CLIP, navega hasta la posición más probable del objeto y verifica visualmente la detección con YOLOE. Las integraciones críticas completadas en esta fecha son:

- **Bug A1 resuelto**: `IndexError` en `Navigator.plan_to()` cuando el goal cae exactamente en el borde del mapa de obstáculos — se añade un clamp $[0, h-1] \times [0, w-1]$ sobre las coordenadas del goal en el mapa recortado.
- **Bug A2 resuelto**: umbral de confianza YOLOE subido de 0,25 a 0,40 para reducir falsas detecciones.
- **Frame de detección congelado**: cuando YOLOE confirma el objeto, la ventana *1st person* muestra el frame anotado (con bbox) hasta que se inicia una nueva búsqueda, en lugar de volver al feed en vivo.
- **Paso B implementado**: fórmula de calidad de región añadida a `postprocess_heatmap()` — los componentes se ordenan por calidad en lugar de por score bruto.

9.2. Flujo completo del heatmap (implementado)

El heatmap es la representación 2D de la probabilidad semántica de encontrar el objeto buscado en cada celda del mapa. El flujo completo, implementado en `application/interactive_object_nav.py`, es el siguiente:

Paso 1 — Scores CLIP por vóxel

Cada vóxel del mapa 3D tiene asociado un vector de scores CLIP de dimensión $|\mathcal{C}|$ (número de categorías). Para una query dada, se extrae el score $s_i \in [0, 1]$ de cada vóxel i para la categoría objetivo. Solo se retienen los vóxeles en los que esa categoría tiene el argmax y el score supera un umbral mínimo ($s_i > 0,3$).

Paso 2 — Proyección 2D (max-pool por columna)

Los scores 3D se proyectan sobre el plano del suelo tomando el máximo por celda (r, c) . El resultado es un heatmap flotante de tamaño $gs \times gs$ (por defecto $gs = 960$).

Paso 3 — Difusión por distancia

Se aplica una difusión espacial ligera basada en `distance_transform_edt`: las celdas adyacentes a celdas activas reciben score propagado con una caída lineal ($\times 0,3$ por celda). Las celdas con score $< 0,15$ se anulan, limitando la difusión a una franja de ~ 3 celdas (15 cm).

Paso 4 — Suavizado Gaussiano

Se aplica un *Gaussian blur* con kernel 5×5 y $\sigma = 1,0$ para suprimir ruido de alta frecuencia y reforzar las regiones espacialmente coherentes.

Paso 5 — Umbral relativo y componentes conectados

Se binariza el heatmap suavizado con un umbral relativo:

$$t = \text{rel_thresh} \times \text{máx}(\text{heatmap suavizado}), \quad \text{rel_thresh} = 0,5$$

Sobre la máscara binaria resultante se calculan los componentes conexos (8-conectividad, `cv2.connectedComponents`). Para cada componente r se extraen: área, score máximo, score medio, score suma, bounding box y centroide.

Paso 6 — Scoring interno y filtro relativo

Cada componente recibe un score interno para decidir si se descarta:

$$\text{score}(r) = \text{mean_val}(r) \cdot \log(1 + \text{area}(r))$$

Se descartan los componentes cuyo score sea inferior al 20 % del mejor:

$$\text{score}(r) \geq 0,2 \cdot \max_j \text{score}(j)$$

Esto elimina islas pequeñas y dispersas sin reglas manuales por categoría.

Paso 7 — Fórmula de calidad (Paso B)

Sobre los componentes supervivientes se calcula una métrica de calidad que combina tres dimensiones complementarias:

$$\text{quality}(r) = 0,3 \cdot \widehat{\text{area}}(r) + 0,4 \cdot \bar{s}(r) + 0,3 \cdot \delta(r)$$

donde:

- $\widehat{\text{area}}(r) = \text{area}(r) / \max_j \text{area}(j)$ — área normalizada respecto al componente mayor (0–1).
- $\bar{s}(r) = \text{mean_val}(r)$ — score CLIP medio de los vóxeles del componente (ya en 0–1).

- $\delta(r) = \text{area}(r) / (w_r \cdot h_r)$ — densidad: fracción de celdas activas dentro del bounding box del componente (0–1).

Los componentes se ordenan por *quality* de mayor a menor. El de mayor calidad es el goal primario de navegación; el siguiente candidato se usa como ruta alternativa.

Paso 8 — Selección del goal

`select_safe_goal_from_path()` recorre el camino planificado hacia el standoff del componente y elige el último waypoint que tiene score de heatmap no nulo, garantizando que el robot se detiene frente al objeto con visibilidad directa.

9.3. Resultados recientes — 24 de abril de 2026

Durante la iteración del 24 de abril se ha ejecutado una evaluación en dos fases sobre las tres escenas HSSD ya integradas: 102344193_0, 102344280_0 y 108736884_177263634_4.

Fase 1 — comparación de heatmaps. Se generó una batería *stressed_heatmap* y se expandió hasta 500 queries por escena. El objetivo era aislar el efecto del postprocesado del heatmap sin mezclarlo con el comportamiento del orquestador. El resultado agregado fue:

- **Ganador global:** postprocessed.
- **Escena 102344193_0:** hit@1 empata (0,43 → 0,43), pero *mass_in_expected_ratio* mejora (0,374 → 0,400).
- **Escena 102344280_0:** mejora clara en hit@1 (0,316 → 0,414), mejora en *mass_in_expected_ratio* (0,281 → 0,310) y reducción de *wrong_room_mass_ratio* (0,577 → 0,551).
- **Escena 108736884_177263634_4:** mejora moderada en hit@1 (0,326 → 0,352), mejora en *mass_in_expected_ratio* (0,276 → 0,298) y ligera reducción de masa en habitaciones incorrectas.

Interpretación. El postprocesado **no mejora todas las métricas geométricas** del heatmap: hit@5 empeora en dos escenas, iou_topmass50 empeora en las tres y *n_components* permanece igual. Sin embargo, sí mejora las métricas que mejor capturan la utilidad práctica del mapa para navegación semántica:

- **hit@1:** la primera hipótesis cae más veces en la zona correcta.
- **mass_in_expected_ratio:** la masa se concentra mejor donde debería.
- **wrong_room_mass_ratio:** baja la masa derramada en habitaciones incorrectas en dos de las tres escenas.

La conclusión práctica es que **el postprocesado sí merece la pena** como configuración base, no porque “gane en todo”, sino porque mejora la calidad de la *hipótesis principal*, que es lo que realmente utiliza el sistema al elegir el siguiente candidato.

Fase 2 — comparación de orquestadores con heatmap fijo. Fijado el heatmap ganador (Hp), se compararon únicamente los dos métodos del eje de orquestación: Ob_Hp (baseline) y Oe_Hp (executor). La batería *stressed_orchestrator* se expandió hasta 100 queries por método y por escena.

El agregado cross-scene da:

Método	SR	Wrong Visits	Mean Pose Updates	Early Stop Rate
Ob_Hp	0.6667	0.1600	382.44	0.1000
Oe_Hp	0.6167	1.1800	431.58	0.0267

Interpretación. En este batch estresado, el executor **no supera** al baseline una vez fijado el heatmap postprocesado. Aunque mejora ligeramente algunas métricas intermedias (CT2R, Rooms Before Success, Early Stop Rate), pierde en las métricas principales:

- **menor SR global** (0,6167 frente a 0,6667),
- **muchas más visitas erróneas** (1,18 frente a 0,16),
- **más coste de navegación** (431,58 frente a 382,44 pose updates de media).

Caveats. La escena 102344193_0 aporta menos evidencia que las otras dos porque su batería estresada original solo contenía 7 queries únicas y hubo que expandirla por repetición para llegar a 100/500 muestras. Además, el wrapper del orquestador produjo un `aggregate_cross_scenes_pruned` vacío por un bug de postprocesado del agregado; los resultados válidos son los del `aggregate_cross_scenes.csv` bruto, no los del fichero `pruned`.

Conclusión metodológica. La separación en dos fases tiene sentido y ha resultado informativa:

1. **Fase 1:** Hp se adopta como heatmap base porque mejora la hipótesis principal y reduce ruido útil.
2. **Fase 2:** una vez fijado Hp, el baseline Ob_Hp sigue siendo, por ahora, el método más sólido y equilibrado.

En otras palabras: **el postprocesado aporta valor real**, mientras que **el executor todavía no justifica su sobre coste** en esta batería estresada.

9.4. Próximos pasos

Paso C — Métodos baseline M1 y M3 (1–2 sesiones)

- **M1 — Navegación aleatoria:** el robot elige celdas navegables al azar como goals sucesivos. Sirve como cota inferior de rendimiento en la evaluación.
- **M3 — Ranking por calidad:** usa directamente el campo `quality` ya disponible en `kept_components` para ordenar los goals y navegar en orden decreciente de calidad. Pre-requisito directo: Paso B (ya implementado).

Paso D — Razonamiento LLM iterativo con contexto (2–3 sesiones)

Inspirado en MoMa-LLM (RSS SemRob 2024), sin usar código propietario de Toyota. El robot mantiene un registro de contexto de búsqueda: habitaciones visitadas, candidatos probados y resultado (encontrado / no encontrado). En cada iteración fallida, este contexto se pasa a un LLM (GPT-4o o similar) con un prompt del tipo:

«Busco {objeto}. He visitado {habitaciones}. He probado los candidatos {lista} sin éxito. ¿En qué tipo de habitación o zona debería buscar ahora?»

El LLM devuelve una sugerencia de zona o tipo de habitación, que se mapea a una región del mapa usando el mapa de habitaciones ya disponible. Implementación desde cero, sin dependencias del repositorio de MoMa-LLM.

Paso E — Script de evaluación cuantitativa (1 sesión)

Métricas estándar de navegación semántica:

- **SR** (Success Rate): fracción de episodios en los que YOLOE confirma la detección.
- **SPL** (Success weighted by Path Length): SR ponderado por la eficiencia del camino recorrido.
- **Wrong Visits**: número de goals visitados sin detección positiva antes del éxito.
- **Time to Find**: número de pasos de simulación hasta la confirmación.

Paso F — M4: scoring híbrido + look-while-move (2 sesiones)

Combinar score VLMMap con distancia al goal en una función única:

$$goal_score(g) = \alpha \cdot quality(g) - (1 - \alpha) \cdot \frac{d(g)}{d_{\text{máx}}}$$

Añadir detección YOLOE asíncrona durante el desplazamiento (*look-while-move*): el robot verifica con YOLOE cada k pasos mientras navega, sin detener la ejecución.

Paso G — M5: viewpoints + exploración de fronteras (3 sesiones)

Generar candidatos de viewpoint alrededor de cada componente del heatmap (en lugar de un único standoff). Añadir exploración de fronteras para descubrir zonas no mapeadas si todos los viewpoints fallan.

Paso G.5 — Objetos no-VLMMap abiertos con LLM + validación (1 sesión)

Objetivo. Preparar el sistema para búsquedas de nivel 3 (laptop, book, soap, vase, candle, basket, printer, teapot) sin depender todavía de ROS. La implementación debe caber en una sesión de trabajo y dejar la semántica lista para probarse mañana.

Arquitectura mínima.

1. **Normalizador de alias.** Añadir una capa pequeña para resolver equivalencias obvias: computer→laptop, books→book, tea pot→teapot, soap dispenser→soap.
2. **Prompt estructurado al LLM.** En lugar de devolver texto libre, pedir un JSON con:

```
{
  "canonical_target": "laptop",
  "query_mode": "surrogate",
  "surrogate_categories": ["desk", "table", "sofa", "bed"],
  "likely_rooms": ["office", "bedroom"],
  "yoloe_label": "laptop",
  "aliases": ["computer", "notebook"]
}
```
3. **Validador determinista.** Filtrar la lista de surrogate_categories contra get_categories("hssd") y quedarse solo con muebles VLMMap ejecutables.
4. **Fallback auditado.** Si el LLM falla o devuelve una salida vacía, usar object_priors.py como respaldo.
5. **Verificación YOLOE.** El target enviado a YOLOE debe ser el canonical_target, no el surrogate.

Archivos candidatos para tocar.

- `vlmaps/utils/llm_utils.py`: prompt y parseo del JSON.
- `vlmaps/utils/object_priors.py`: fallback y alias auditados.
- `application/interactive_object_nav.py` e `interactive_object_nav_executor.py`: integración del target canónico y la lista validada de surrogates.
- `vlmaps/policy/strategic_policy.py`: solo si hace falta propagar el target canónico al snapshot de estrategia.

Plan exacto para una sesión.

1. Crear una función nueva del estilo `resolve_open_vocab_target(target, available_categories)`.
2. Llamarla únicamente cuando el target no pertenezca al vocabulario VLMap.
3. Validar que los surrogates finales sean categorías HSSD navegables.
4. Mantener `canonical_target` como etiqueta de YOLOE.
5. Registrar en el log final: target original, target canónico y lista final de surrogates.
6. Probar manualmente la shortlist aprobada.

Criterio de salida. El sistema debe seguir funcionando para queries directas VLMap y, además, ser capaz de buscar la shortlist anterior usando solo muebles surrogate válidos y verificación final con YOLOE.

Paso H — Wrapper ROS 1 Noetic / Toyota HSR (2–3 sesiones)

Adaptar el pipeline para el robot real Toyota HSR mediante un nodo ROS que exponga la misma interfaz de navegación semántica. Requiere driver de cámara RGBD, localización con mapa real y adaptación del sistema de coordenadas.

10. Diario de desarrollo

13 de abril de 2026

1. **Bug protocolo stdout/stderr YOLOE:** `_ensure_mobileclip()` imprimía en stdout interceptando el token `READY`; corregido a `sys.stderr`. `start()` ahora lee hasta recibir `READY`.
2. **mobileclip_blt.ts:** búsqueda en múltiples rutas candidatas; eliminación automática de versión corrupta (< 100 MB).
3. **Captura stderr del worker:** crashes visibles en consola.
4. **Dirección de cara definitiva:** $\arctan 2(-\Delta y, -\Delta x)$, signo consistente con el planificador.
5. **Sin teletransporte:** `plan_path_only()` + camino 800 ms antes de ejecutar.
6. **Hydra corregido:** `scene_dataset_config_file` sin prefijo + en ejecución y documentación de `menu.sh`.
7. **Repositorio GitHub limpio:** `mobileclip_blt.ts` (571 MB) eliminado del historial con `git-filter-repo`; push operativo.
8. **Bugs corregidos:** `IndexError` boundary (clamp en `plan_path_only()`) + umbral YOLOE subido a 0,40.

9. **Fórmula de calidad:** implementada en `postprocess_heatmap()`; componentes ordenados por `quality`.

7 de junio de 2026: validación semántica multiclase del VMap ROS/Gazebo

Se cierra la validación del flujo de captura ROS/Gazebo a VMap con una prueba sin interfaz gráfica que replica el criterio nativo de VLMaps: inicializar un vocabulario multiclase con `init_categories(mp3dcat)`, añadir las categorías solicitadas que no estén en MP3D y asignar cada vóxel a la categoría que gana el `argmax` frente al resto de clases y la clase de fondo `other`. Este criterio sustituye al diagnóstico anterior basado en el score crudo de una única categoría, que no era discriminativo por la colinealidad de CLIP y hacía parecer equivalentes mapas semánticamente distintos.

1. **Nuevo modo sin interfaz gráfica:** se añade `vlmap_ros_export.index_validation`, ejecutable con `python -m`. Carga el VMap con la misma función que el exportador ROS, proyecta máscaras 3D a 2D, escribe `.npy` compatibles con el consumidor de heatmaps, genera overlays `.png` con Matplotlib en modo `Agg` y guarda `metrics.json` con áreas, centroides, IoU y solapes por pares.
2. **Integración en menú:** la opción 3 del menú de `tfg-sim` mantiene el export crudo existente y añade la subopción `i)` para volcar índices semánticos multiclase en `/shared/heatmaps/<world>/index`.
3. **Resultado en captura ROS/Gazebo:** sobre `/shared/captures/small_house/run_20260607_174653` (474 frames), el solape medio entre categorías de habitaciones distintas es **0.00010** y el máximo **0.00126**. Las máscaras son prácticamente disjuntas: cama en la zona de dormitorio, fregadero en la zona de cocina, sofá y sillas en la zona común y `toilet` ausente.
4. **Comparación HSSD:** sobre `102344280_0`, el solape medio es **0.0120** y el máximo **0.1655**, correspondiente al par `table-chair`, una superposición esperable por la proximidad entre mesa y sillas. El resto de clases cae en zonas coherentes de baño, dormitorio, cocina y salón.

Mapa	Celdas ocupadas	IoU medio	IoU máximo	Par máximo
small_house ROS/Gazebo	25 629	0.00010	0.00126	refrigerator-sink
HSSD 102344280_0	21 453	0.0120	0.1655	table-chair

Cuadro 1: Validación del índice semántico multiclase. El mapa capturado desde ROS/Gazebo discrimina categorías de habitaciones distintas con solapes bajos y separación comparable al mapa HSSD de referencia.

24 de abril de 2026

1. **Separación formal de la evaluación en dos fases:** se abandona la lectura monolítica del harness `2x2` y se decide evaluar por separado (i) la calidad del heatmap y (ii) el comportamiento del orquestador. El objetivo es aislar causas y poder justificar en la memoria del TFG qué parte de la mejora viene del postprocesado del mapa semántico y qué parte viene realmente de la política de decisión.
2. **Nueva batería *stressed_heatmap*:** se filtran queries con etiquetas como `multi_instance`, `ambiguous_room` y `explicit_instance` para forzar escenarios donde el heatmap raw produce más picos espurios y el postprocesado debería diferenciarse. La batería se expande hasta **500 queries por escena** para dar estabilidad estadística al análisis offline.
3. **Resultado de la fase heatmap:** el ganador agregado sobre las tres escenas HSSD es `postprocessed`. No gana por una mejora uniforme de todas las métricas geométricas, pero sí en las que mejor capturan la utilidad práctica para navegación:

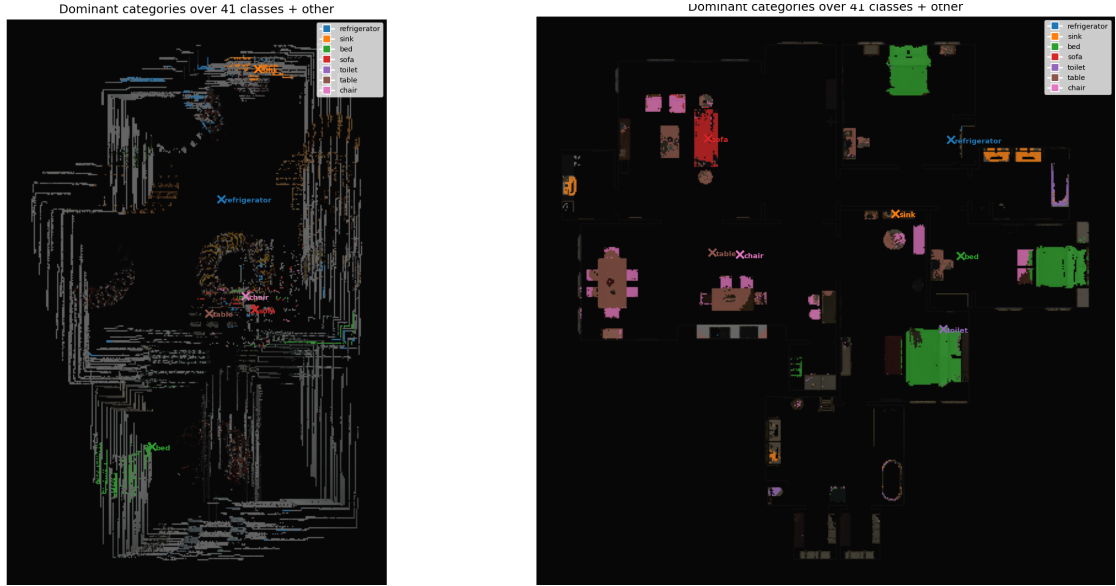


Figura 11: Resumen dominante por celda para el mapa capturado en `small_house` (izquierda) y para HSSD 102344280_0 (derecha). Cada color representa la categoría que gana el índice multiclase frente al resto del vocabulario y el fondo.

- `hit@1`: empata en 102344193_0 y mejora en 102344280_0 (0,316 $\rightarrow$ 0,414) y 108736884_177263634_4 (0,326 $\rightarrow$ 0,352).
- `mass_in_expected_ratio`: mejora en las tres escenas.
- `wrong_room_mass_ratio`: desciende en dos de las tres escenas, reduciendo la masa derramada en habitaciones incorrectas.

En cambio, `hit@5` empeora en dos escenas, `iou_topmass50` empeora en las tres y `n_components` permanece estable. La conclusión adoptada es que `Hp` sí merece la pena porque mejora la *hipótesis principal*, no porque optimice todas las vistas geométricas del mapa.

4. **Nueva batería *stressed_orchestrator***: una vez fijado el heatmap ganador, se comparan únicamente `Ob_Hp` y `Oe_Hp` sobre queries donde el razonamiento estratégico debería importar más. La batería se expande hasta **100 queries por método y escena**.
5. **Resultado de la fase orquestador**: el baseline `Ob_Hp` supera al executor `Oe_Hp` en el agregado cross-scene del batch estresado:

Método	SR	Wrong Visits	Mean Pose Updates	Early Stop Rate
<code>Ob_Hp</code>	0.6667	0.1600	382.44	0.1000
<code>Oe_Hp</code>	0.6167	1.1800	431.58	0.0267

Aunque el executor mejora algunas métricas intermedias, pierde en las métricas principales y además incurre en muchas más visitas erróneas y mayor coste de navegación.

6. **Configuración base fijada tras la evaluación**: con la evidencia reunida hasta esta fecha, la combinación más sólida para seguir avanzando pasa a ser:
 - heatmap: `postprocessed`,
 - orquestador: `baseline`,
 - umbral YOLOE: 0.30.

Esta decisión sustituye a la narrativa previa de “el executor debería ganar” por una lectura

más honesta: el postprocesado aporta valor real y el executor, de momento, no justifica su complejidad adicional.

7. **Cambio de foco estratégico:** antes de migrar el proyecto a ROS 1, se decide cerrar una capa de semántica abierta sobre HSSD para objetos no-VLMap. La idea es validar primero, en simulación controlada, un flujo *LLM propone* $\rightarrow$ *validador filtra* $\rightarrow$ *YOLOE confirma*, aplicable a objetos concretos como `laptop`, `book`, `bottle`, `teapot`, `printer`, etc.
8. **Documentación del plan siguiente:** se añade al estado del proyecto una nueva etapa intermedia antes de ROS para soportar objetos abiertos no presentes en el vocabulario base de VLMaps. La intención es endurecer la capa semántica y llegar a la migración con un núcleo más maduro, evitando mezclar de golpe problemas de ROS, TF, sensores y razonamiento semántico.

14 de abril de 2026

1. **Reestructuración del documento de proyecto:** se redefine la finalidad del proyecto con foco en razonamiento por habitaciones.
2. **HSSD confirmado como dataset principal:** MP3D queda como soporte secundario; LabelMe diferido.
3. **Arquitectura room-aware diseñada:** 8 fases (A–H) con representación de estado por habitación, scoring de selección de room, acciones de alto nivel y rol estratégico del LLM.
4. **Roadmap corregido:** la escalera M1–M5 se reemplaza por la arquitectura room-aware como prioridad central. Baselines y evaluación pasan a fase posterior.
5. **Fase A implementada:** `SemanticSceneRoomProvider` construido con transformación completa `Habitat` $\rightarrow$ `grid` (`hab_to_grid`). Polígonos rasterizados en espacio grid. Cada componente del heatmap anotado con su habitación. Navegación a habitación por nombre funcional (centroide directo, sin heatmap ni YOLOE). Verificación visual de llegada a habitación omitida en simulación (coordenadas exactas); se contempla reactivarla para robot real con localización imperfecta.
6. **Parsing semántico por niveles:** identificado como trabajo futuro necesario. El parsing actual separa instrucciones compuestas en targets independientes; se requiere un parser que interprete habitación objetivo + objeto dentro de ella como una sola instrucción estructurada.

15 de abril de 2026

1. **Fase B completada:** `RoomState` y `SearchState` implementados en `vlmaps/utils/search_state.py`. Cada `RoomState` registra `explored_ratio`, `objects_seen`, `candidates_tried/confirmed`, `times_visited`, `target_relevance` y `target_found_here`. Se construye una instancia por target al inicio de cada instrucción y se actualiza incrementalmente durante la navegación y la verificación YOLOE. Al final de cada instrucción se imprime un resumen completo del estado de búsqueda.
2. **Fase C completada:** módulo `vlmaps/utils/room_priors.py`. Implementa `compute_room_priors(query, known_rooms, objects_seen_by_room)` fusionando tres fuentes con pesos $a = 0,5$, $b = 0,3$, $c = 0,2$: (a) LLM GPT-4o-mini con llamada estructurada al inicio de cada búsqueda de objeto; (b) tabla manual de ~ 35 objetos con scores por tipo de habitación; (c) evidencia de co-ocurrencia basada en `objects_seen` por habitación. El resultado normalizado se almacena en `RoomState.target_relevance` y se muestra como traza de depuración antes de cada consulta.

3. **Navegación solo a candidatos del heatmap:** eliminado el uso de `get_standoff_pos()` (que usaba `labeled_map_cropped`). Ahora el robot navega directamente al centroide del mejor componente de `kept_components` (filtrado real por `argmax + score > 0,3`). Esto evita que el robot viaje a zonas con señal VLMap pero sin activación real en el heatmap.
4. **Filtrado de objetos navegables:** la lista de objetos mostrada antes de cada consulta usa el mismo filtro que `compute_heatmap` (vóxeles con `argmax` ganador y `score > 0,3`, mínimo 10 vóxeles), garantizando coherencia entre lo que se muestra y lo que el sistema puede encontrar.
5. **Correcciones de navegación:** (a) `n_actions == 0` tratado como “ya en el destino” en vez de “sin camino”, corrigiendo el caso post-llegada a habitación; (b) eliminado el bloque de reintento por paso estrecho (*narrow-passage retry*) que generaba más falsos positivos que beneficios; (c) umbral YOLOE subido a 0,3 y dilatación de obstáculos a 3 iteraciones (≈ 15 cm por lado).
6. **Tipos canónicos de habitación (*room_type* vs *room_instance*):** HSSD codifica instancias repetidas de un mismo tipo de habitación con sufijos numéricos (`bathroom.001`, `bathroom.002`, etc.). Se añade `canonical_room_type()` en `room_priors.py` que elimina el sufijo `.NNN` mediante regex (`re.sub(r'\.d+$', '', s)`), y `compatible_room_types()` que devuelve el conjunto de tipos canónicos con `prior > 0,05`. Estas funciones permiten razonar sobre familias de habitación independientemente del índice de instancia.
7. **Comportamiento *local-first* al navegar (*room gate*):** nueva función `select_best_candidate(kept_c, current_room, query_priors, tried_centroids)` en `interactive_object_nav.py`. Si el robot ya se encuentra dentro de una instancia de habitación cuyo tipo canónico es compatible con el objeto buscado (e.g. robot en `bathroom.001`, objeto `sink`), el sistema prioriza los candidatos del heatmap situados en esa misma instancia, aplicando un bonus de +0,35 a su calidad efectiva. El cambio de habitación sólo se produce si (a) no quedan candidatos locales sin explorar, o (b) la calidad externa supera la local en más de un 40 % tras el bonus. El conjunto `_tried_centroids` en `SearchState` registra los centroides ya visitados para evitar ciclos.
8. **Navegación con margen de seguridad aumentado:** la dilatación de obstáculos en `habitat_lang_robot.py` se eleva a 5 iteraciones (≈ 25 cm por lado). La función `select_safe_goal_from_p` aumenta el radio mínimo de *clearance* de 3 a 5 celdas y adopta estrategia de máxima holgura: reúne *todos* los puntos del camino que cumplen el radio y selecciona el de mayor distancia al obstáculo más cercano (en vez del primero válido caminando hacia atrás). Si la pasada estricta no devuelve candidatos, una segunda pasada suaviza el umbral y vuelve a seleccionar por máxima holgura.
9. **Métricas de seguridad por camino:** nueva función `_compute_path_safety(path_cells, obs_map)` basada en `distance_transform_edt`. Registra longitud, holgura mínima y media, penalización acumulada por celdas con `clearance < 8` y coste seguro = longitud + $0,5 \cdot$ penalización. Los valores se imprimen en trazas `[safety]` tras cada planificación, facilitando el diagnóstico de rutas peligrosas y la futura migración a ROS/HSR.
10. **Corrección Bug 1 — comandos de habitación con validación real de llegada:** los comandos de tipo `kitchen`, `dining room`, etc. fallaban porque el sistema navegaba al centroide geométrico del polígono de la habitación (a menudo sobre un obstáculo), el planificador lo desplazaba a un nodo visgraph cercano (frecuentemente en otra habitación) y después declaraba éxito sin comprobar dónde estaba realmente el robot. Se implementa `find_reachable_room_goal()`, que enumera todas las celdas navegables del interior de la habitación objetivo, las ordena por holgura respecto a obstáculos y devuelve los top-*K* candidatos seguros. Tras la ejecución del camino se comprueba la habitación real con

`get_room_at_cell()` y se valida mediante `_room_instance_matches()`, que acepta tanto coincidencia exacta como de tipo canónico (`kitchen.001` satisface el comando `kitchen`). Si la primera meta falla, se reintenta con los candidatos alternativos antes de declarar fracaso. Solo se imprime “Arrived at room X” cuando la comprobación pasa.

11. **Corrección Bug 2 — evidencia del heatmap domina en consultas directas:** para consultas de objeto con señal real en el heatmap (p.ej. `sofa`), los priors de habitación dependían excesivamente de los priors lingüísticos (LLM + tabla manual), ignorando que los componentes del heatmap ya indicaban en qué habitación está el objeto. Además, nombres de habitación específicos de la escena como `tv` no coincidían con los fragmentos estándar (`living`) de la tabla manual. Se añade la tabla `_SEMANTIC_ALIASES` que mapea nombres atípicos a familias semánticas (`tv` → `living`, `laundryroom` → `laundry`, etc.) y se aplica en `_manual_prior()` y `_evidence_prior()` mediante `_normalize_room_for_priors()`. Se añade `compute_heatmap_room_evidence()` que agrega la calidad de componentes por instancia de habitación y normaliza a $[0, 1]$. Se extiende `compute_room_priors()` con los parámetros opcionales `heatmap_evidence` y `query_type`: las consultas directas usan pesos $0,65 \cdot \text{heatmap} + 0,20 \cdot \text{manual} + 0,15 \cdot \text{LLM}$, mientras que las indirectas mantienen $0,45 \cdot \text{LLM} + 0,30 \cdot \text{manual} + 0,25 \cdot \text{evidencia escena}$. En el bucle de navegación, tras anotar los componentes con su habitación, se recalculan los priors con evidencia real antes de seleccionar el candidato, de modo que `sofa` con tres componentes en `tv` produce un prior $> 0,86$ para `tv`.
12. **Corrección Bug 3 — puerta de seguridad bloquea ejecución de rutas inseguras:** las métricas de seguridad [`safety`] existían pero eran puramente informativas: el sistema ejecutaba igualmente rutas con `goal_cl=0.0` o `min_cl=0.0`. Se añaden umbrales duros configurables `_MIN_GOAL_CLEARANCE=3,0` celdas y `_MIN_PATH_CLEARANCE=1,0` celdas. Antes de ejecutar cualquier camino a un objeto, se evalúan dichos umbrales: si el goal o la ruta no los superan, el centroide se marca como intentado en `_tried_centroids` y se prueban hasta tres candidatos alternativos. Si ningún candidato supera los umbrales, la consulta se omite con mensaje explícito. La seguridad ya no es anotación, sino control de flujo.
13. **Corrección Problema 1 — metas de aproximación seguras alrededor del centroide de componente:** el pipeline anterior planificaba hasta el centroide del heatmap (a menudo situado sobre un obstáculo), ejecutaba el camino completo y luego recorría el camino hacia atrás buscando un buen punto de observación. Si el centroide caía dentro de un obstáculo, el planificador visgraph producía rutas erróneas. Se añade `find_safe_candidate_approach_goal` (`safe_obs_map`, `robot_pos`) que muestrea posiciones de aproximación en un anillo de 8–25 celdas alrededor del centroide, filtra por navegabilidad en `_safe_obs_map` y holgura mínima ≥ 3 celdas, y ordena por holgura descendente y proximidad al robot. La meta de navegación se elige directamente del mejor candidato del anillo; el camino hacia atrás solo se activa como respaldo si el anillo queda vacío.
14. **Corrección Problema 2 — planificador global atravesaba paredes:** causa raíz doble. (a) La dilatación de obstáculos con 5 iteraciones (bilateral ≈ 10 celdas) cerraba los pasillos estrechos de HSSD (~ 10 celdas de ancho) desconectando habitaciones en el visgraph. Corregido volviendo a 3 iteraciones (≈ 6 celdas). (b) El método de recuperación de goal en espacio de obstáculo usaba `G.closest_point()` (holgura = 0) con `get_nearby_position()` como fallback, que devuelve `None` si las cuatro diagonales están bloqueadas; `vg.Point(None[0])` genera coordenadas inválidas $(-1, 96)$ y `G.shortest_path` produce una línea recta a través de paredes. Ambas funciones se reemplazan por `_snap_to_nearest_free(p, obstacles, min_clearance)` en `navigation_utils.py`, que usa `distance_transform_edt` para encontrar la celda libre más cercana con holgura suficiente: el resultado está siempre garantizado dentro del espacio navegable del mapa del visgraph. Se añaden también registros de diagnóstico [`planner`] `Start navigable: ... Goal navigable: ...` y almace-

namiento del mapa dilatado completo como `robot._safe_obs_map` para que el resto del sistema use la misma referencia que el visgraph. Las funciones `find_reachable_room_goal()` y `find_safe_candidate_approach_goals()` se actualizan para usar `robot._safe_obs_map` en lugar del mapa de navmesh crudo, eliminando la discordancia entre validación de metas y planificador.

15. **Corrección Problema 3 — escudo de colisiones local en ejecución:** `execute_nav_replay()` acepta ahora el parámetro opcional `dist_map` (transformada de distancias del mapa seguro dilatado, precalculada una vez antes de cada ejecución). Antes de cada acción `move_forward`, el escudo predice la siguiente celda de cuadrícula a partir de la posición actual `curr_pos_on_map` y el ángulo `curr_ang_deg_on_map` del robot. Si la holgura en esa celda es inferior a `STOP_CL = 2,0` celdas, la ejecución se interrumpe inmediatamente y se activa la recuperación; si la holgura está entre `STOP_CL` y `SLOW_CL = 5,0` celdas se emite un aviso sin detener el movimiento. El escudo opera sobre el mismo mapa dilatado que el visgraph, de modo que «celda segura» tiene idéntico significado en planificación y en ejecución.
16. **Navegación one-shot — eliminados reintentos y recuperación:** el escudo de colisiones usaba el mapa dilatado (`_safe_obs_map`) para calcular la transformada de distancias, lo que hacía que el centro de un pasillo de ~ 10 celdas tuviera clearance ≈ 2 en el mapa dilatado y disparara paradas falsas al cruzar puertas. El escudo pasa a usar `robot.map.obstacles_map` (mapa crudo sin dilatar), que refleja el espacio físico real; los umbrales se ajustan a `STOP_CL = 1,0` y `SLOW_CL = 2,5` celdas. Adicionalmente, se elimina la llamada a `nav_recovery_and_replan()` tras un fallo de ejecución y el bucle de reintentos con metas alternativas en comandos de habitación. La navegación es ahora estrictamente one-shot: se planifica una vez, se ejecuta una vez y se reporta el estado real de la habitación; si la ejecución se interrumpe, el robot continúa desde la posición actual sin ruta nueva.

16 de abril de 2026

1. **Separación explícita entre inflación de planificación y escudo de ejecución:** el planificador global utiliza el mapa dilatado (`_safe_obs_map`, 3 iteraciones de dilatación binaria, ≈ 15 cm por lado) para trazar rutas con suficiente margen. El escudo de ejecución opera exclusivamente sobre la transformada de distancias del mapa crudo (`robot.map.obstacles_map`), sin dilatar. Esto garantiza que los pasillos estrechos de HSSD (~ 10 celdas de ancho, clearance real ≈ 5 celdas) no sean bloqueados falsamente por el escudo. La constante `_PLAN_DILATION_ITERS = 3` se declara de forma explícita en `habitat_lang_robot.py` y se imprime al inicio de la sesión junto con el mensaje: «*global planner only; execution shield uses raw map*».
2. **Modo de travesía de puertas (*doorway mode*):** antes de cada `move_forward`, el escudo mide las clearances perpendiculares a la derecha y a la izquierda del robot en la pose actual (`_side_l`, `_side_r`). Si ambas caen por debajo de `DOORWAY_SIDE_TH = 5,0` celdas y la clearance frontal supera `DOORWAY_FRONT_MIN = 0,5` celdas, se activa el modo de paso estrecho. En ese modo el umbral de parada dura se relaja a `STOP_CL_DOOR = 0,5` celdas (frente a 1,0 en espacio abierto) y cada paso se registra como «*cautious doorway traversal*». La transición ON/OFF se imprime explícitamente. El planificador puede enrutar libremente por puertas válidas; el escudo deja cruzarlas sin falsas paradas.
3. **Escudo sensible a la huella completa del robot:** el chequeo anterior solo evaluaba un punto (celda frontal prevista). Ahora se evalúan tres puntos de la huella en la pose predicha: *front-center*, *front-left* y *front-right* (desplazados lateralmente `ROBOT_HW = 2` celdas respecto al eje de avance). Se registra por separado `front clearance`, `left clearance`, `right clearance` y `footprint min clearance`. En espacio abierto, si algún lateral cae por debajo de `SIDE_STOP_CL = 1,5` celdas, el paso se bloquea. En modo puerta, el criterio de bloqueo

es solo el mínimo de la huella frente a `DOORWAY_FRONT_MIN`.

4. **Puerta de entrada a habitación para confirmación YOLOE (*room-entry gate*):** hasta ahora el sistema podía declarar éxito inmediatamente si YOLOE detectaba el objeto desde la puerta antes de haber cruzado el umbral. Se añade una comprobación en los stages 0 y 1 de YOLOE: si el robot está en una habitación distinta de la que contiene el componente del heatmap objetivo (`best_comp['room']`) y la distancia al centroide del candidato supera 20 celdas (≈ 1 m), la detección se marca como *tentativa* (`_yoloe_confirmed = False`) y la búsqueda continúa con el stage siguiente. Solo se acepta como éxito definitivo si el robot ha entrado en la habitación correcta o se encuentra a menos de 20 celdas del candidato. Se registra en consola «*Object visible but room not yet entered → tentative only*» o «*Room entry confirmed*» según el caso.
5. **Centrado de trayectorias en pasos estrechos mediante eje medial local:** el visgraph seguía generando caminos geoméricamente válidos pero sesgados hacia uno de los lados del estrechamiento, lo que hacía que el escudo de ejecución bloquease pasos por clearance lateral insuficiente aun estando en modo de paso estrecho. El primer intento de corrección solo recentraba vértices ya existentes y no bastaba cuando el estrechamiento caía en mitad de un segmento largo del visgraph. Se añade un postprocesado `center_path_on_medial_axis()` en `vlmaps/utils/navigation_utils.py`. Tras obtener la ruta del visgraph, se rasteriza cada segmento y, si detecta una franja con clearance $< 4,0$ celdas, inserta un waypoint auxiliar en la zona más estrecha. Ese waypoint, junto con los intermedios originales, se desplaza hacia la celda de mayor holgura dentro de un disco local de radio 6, usando `distance_transform_edt` del mapa *sin* dilatar. El candidato solo se acepta si mantiene línea de visión libre con los waypoints vecino anterior y siguiente sobre el mapa de planificación dilatado, evitando atajos que corten esquinas o crucen paredes. Después se aplica un suavizado conservador de 3 puntos y se conserva la meta final sin desplazar. El resultado es que la ruta cruza por el centro del paso incluso cuando el cuello de botella no coincide con un vértice del visgraph, y el escudo vuelve a comportarse como mecanismo de seguridad, no como corrector sistemático de trayectorias.
6. **Modo de paso estrecho activado por geometría local, no solo por simetría lateral:** en ejecución real aparecía un caso límite al entrar oblicuamente en un marco o cuello de botella: un lateral del footprint caía por debajo de 1,5 celdas mientras el otro seguía aún en espacio relativamente abierto, de modo que el detector antiguo no activaba el modo de paso estrecho y el escudo bloqueaba el avance con la regla de espacio abierto. Se refuerza la detección en `execute_nav_replay()` con tres señales: (1) estrechamiento en la pose actual, (2) estrechamiento en la huella de la pose siguiente y (3) entrada asimétrica, cuando un lateral ya es crítico pero la clearance frontal sigue siendo cruzable. Además se añade histéresis de salida para mantener el modo activo mientras el robot permanezca dentro del cuello de botella. El modo pasa a modelar *pasos estrechos cruzables* en general, no solo puertas perfectamente simétricas entre habitaciones.
7. **Sustitución del *waypoint chasing* por un seguidor de polilínea con *lookahead*:** el aumento de waypoints auxiliares para centrar la trayectoria en pasos estrechos mejoraba la geometría, pero degradaba la ejecución con cientos de micro-correcciones de rumbo, porque el controlador discreto original perseguía cada subgoal por separado con cuantización fija de 5° y 0,1 m. Se densifica ahora la ruta planificada a nivel de celda mediante rasterización de segmentos y se ejecuta con un *path follower* sobre esa polilínea densa. En cada iteración, el robot proyecta su pose sobre la ruta, selecciona un objetivo adelantado (*lookahead*) varios pasos por delante y solo aplica la primera acción discreta necesaria para aproximarse a ese punto. El *lookahead* se reduce automáticamente dentro de pasos estrechos y se mantiene más largo en espacio abierto. Además, las métricas de seguridad y la selección de metas de observación pasan a evaluarse sobre la misma polilínea densificada, de modo que plani-

ficación, validación y ejecución comparten por fin la misma representación geométrica del camino.

8. **Lookahead recortado por visibilidad local del camino:** el primer seguidor con *lookahead* seguía teniendo un fallo importante en pasillos con giro o marcos oblicuos: podía elegir como objetivo un punto varios pasos por delante que ya estaba al otro lado de una esquina no visible desde la pose actual. Como el controlador discreto solo orienta hacia ese objetivo y aplica la primera acción, el robot intentaba cortar la esquina y acababa proponiendo un avance con `front clearance = 0.0`. Se corrige haciendo que el *path follower* no use directamente el *lookahead* preferido, sino el punto más lejano de la polilínea densa que siga siendo alcanzable con línea de visión libre sobre el mapa de planificación dilatado. Así, dentro de cuellos de botella y giros cerrados, el seguidor deja de apuntar “a través de la pared” y se mantiene sobre el tramo realmente visible del camino.
9. **Etiquetas de llegada coherentes con el resultado real de la navegación:** en comandos de habitación se estaba mostrando “*Arrived: ...*” inmediatamente después de que terminara la ejecución del follower, incluso si el robot se había detenido antes del umbral y la comprobación final de habitación fallaba. Esto inducía a error durante la depuración visual. Se corrige la interfaz para que el overlay solo muestre “Arrived” cuando el robot ha alcanzado de verdad la habitación objetivo; en caso contrario, ahora se muestra “Not reached” en navegación a habitaciones o “Stopped before goal” en aproximaciones a objetos.
10. **La referencia de control vuelve a ser la ruta densa, no solo la polilínea compactada:** la versión que seguía únicamente la polilínea geométrica simplificada seguía permitiendo atajos laterales del controlador discreto. En la práctica, el robot podía “llegar más o menos” al final del segmento pero desviándose del eje real del camino, saliéndose hacia paredes o quedándose en la habitación vecina cuando el objetivo estaba cerca de una frontera semántica. Se reintroduce la trayectoria densa rasterizada como referencia de control, pero sin perseguir cada celda individualmente: el follower mantiene *lookahead* sobre esa ruta densa y la usa para medir progreso y error lateral. La polilínea compactada queda como representación del plan, mientras que la ejecución se ancla a la traza detallada que realmente debe seguir el robot.
11. **Banda muerta angular para eliminar micro-correcciones de 5°:** incluso después de volver a seguir la polilínea geométrica, el controlador discreto seguía tendiendo a consumir pequeños giros antes de casi cada avance cuando el error angular hacia el siguiente objetivo era muy bajo. Eso no cambiaba de forma significativa la seguridad, pero sí alargaba innecesariamente la ejecución en pasos estrechos. Se añade una tolerancia angular en el follower: si el objetivo inmediato ya está alineado dentro de un margen moderado, se prioriza `move_forward` en lugar de corregir rumbo con otro giro discreto. La tolerancia es más amplia en zonas estrechas que en espacio abierto. Con ello, se reduce el número de pasos y el patrón de micro-corrección sin introducir nuevos modos especiales de paso por puertas.
12. **El shield preventivo se elimina de la ejecución paso a paso:** el análisis fino mostró que el escudo local estaba mezclando dos nociones incompatibles: clearance sobre un mapa ya dilatado por el radio del agente y probes adicionales alrededor del cuerpo. Eso generaba falsos bloqueos en puertas y convertía la seguridad en el mecanismo principal de decisión, cuando debería ser solo una red de protección. La arquitectura actual elimina el *pre-step shield* y delega la colisión física en la navmesh de Habitat. La robustez pasa a depender de dos piezas más limpias: planificación *clearance-aware* y seguimiento fiel del camino.
13. **Watchdog de no-progreso real en el follower:** además del fallo semántico del shield, aparecía un segundo problema en ejecución: el robot podía quedarse vivo cientos de iteraciones repitiendo prácticamente la misma pose y agotando el *step budget*. Se añade un detector de no-progreso que vigila tanto la celda actual como el índice efectivo de progreso

sobre la polilínea. Si durante varias iteraciones no cambia ni la celda útil ni el progreso útil, la ejecución se aborta pronto. También se detecta el caso en que el controlador no produce acciones válidas repetidamente para el mismo target local. La versión inicial de este watchdog abortaba incorrectamente durante la fase de giro en sitio, porque tomaba “misma celda” como sinónimo de “sin progreso” incluso cuando la orientación seguía cambiando de forma útil hacia el siguiente tramo del camino. Se corrige para que una rotación real también cuente como progreso temporal del follower. Con ello, los fallos pasan a ser rápidos y diagnosticables en lugar de consumir cientos de pasos inmóviles o abortar antes de empezar a avanzar.

14. **Reenganche explícito al camino cuando el robot se sale del eje:** incluso sin *shield*, el follower seguía pudiendo comportarse de forma errática si la banda muerta angular le dejaba avanzar cuando ya se había separado lateralmente del trazado. Se añade una lógica de *off-track reattachment*: la distancia del robot a la ruta densa se mide en cada iteración y, si supera un umbral, el *lookahead* se acorta automáticamente para volver a engancharse al eje del camino antes de permitir más avance agresivo. Además, la banda muerta angular solo puede forzar `move_forward` cuando el error lateral sigue siendo pequeño.
15. **Llegada más estricta en navegación a habitaciones:** la lógica anterior consideraba alcanzado el objetivo si el robot quedaba a pocas celdas del final del camino. Eso era suficiente para aproximaciones a objetos, pero no para comandos de habitación cerca de fronteras semánticas: el robot podía detenerse a 1–2 celdas del goal y seguir clasificado en la estancia vecina. Se introduce una tolerancia de llegada más estricta para navegación a habitaciones, de modo que el follower no dé por completado el camino hasta estar realmente encima del objetivo final o prácticamente superpuesto a él.
16. **Deadband angular menos restrictiva en el follower:** la primera versión de la banda muerta angular seguía siendo demasiado conservadora porque solo permitía sustituir un giro por un avance si el segundo elemento del plan local del controlador ya era `move_forward`. En la práctica, eso dejaba fuera muchos casos útiles en los que el error angular ya era pequeño pero el controlador aún proponía varios giros discretos consecutivos. Se elimina esa condición adicional: ahora el follower prioriza `move_forward` siempre que el error angular esté dentro de la tolerancia y la distancia al objetivo inmediato siga justificando el avance. Esto reduce el patrón “giro-giro-giro-avance” en puertas y pasillos angostos sin cambiar la arquitectura general del controlador.

20 de abril de 2026

1. **Validación en vivo dentro del contenedor de simulación:** en lugar de seguir depurando solo con logs pegados manualmente, se pasa a ejecutar pruebas reales dentro de `tfg-sim` sobre la escena HSSD usada durante el desarrollo (`scene_id=1`). Esto permitió comprobar de forma reproducible tres consultas representativas: navegación a habitación (`kitchen`), búsqueda de objeto con ambigüedad semántica local (`sofa`) y búsqueda de objeto con varios candidatos plausibles (`sink`). La prueba en vivo mostró que `kitchen` seguía fallando aunque el *shield* ya no bloqueaba puertas, mientras que `sofa` y `sink` sí llegaban a confirmarse con YOLOE. Con ello quedó claro que el problema residual principal ya no estaba en la planificación ni en la colisión preventiva, sino en el seguimiento local del camino.
2. **Corrección del bucle `preview produced no action`:** el fallo real de `kitchen` no era una colisión contra paredes, sino un colapso del follower cerca de un tramo concreto del camino, donde el objetivo local quedaba tan cerca que el controlador discreto devolvía repetidamente una previsualización vacía (`progress=58`, `target=59`). El follower seguía reenganchándose a ese mismo índice una y otra vez y acababa abortando sin salir del

cuello. Se corrige en dos pasos. Primero, el objetivo visible de *lookahead* ya no puede ser arbitrariamente cercano salvo cuando el robot está realmente desviado o casi en la meta. Segundo, cuando un target local sigue produciendo una previsualización vacía, el sistema eleva explícitamente un *progress floor* y fuerza al follower a no volver a anclarse en ese mismo punto de la ruta. Con este cambio, la consulta *kitchen* deja de abortar en el mismo progreso y completa la trayectoria hasta quedar clasificada correctamente dentro de la cocina.

3. **La calidad semántica del heatmap deja de premiar blobs minúsculos:** el caso de *sofa* reveló otro problema de calidad. La fórmula anterior para puntuar componentes del heatmap aún permitía que un blob muy pequeño pero extremadamente denso compitiera con una región grande correspondiente al sofá real. En la práctica, el robot podía priorizar un sillón o una activación residual cercana solo porque el componente era compacto, aunque el soporte espacial del sofá fuese mucho mayor. Se reformula la *quality* de cada componente para favorecer de forma más explícita el soporte semántico real (*sum_val*) y el tamaño útil del componente, reduciendo el peso relativo de activaciones pequeñas. Tras este cambio, en la consulta *sofa* el componente grande asociado al sofá pasa a quedar por delante de los blobs minúsculos, evitando que el robot “vaya al sillón” salvo que semánticamente sea de verdad el mejor candidato.
4. **Overlay y visualización más fluidos sin cambiar de backend:** también se revisa la parte puramente visual del sistema. El overlay de navegación mostraba etiquetas del tipo [64/366] ->sofa, pero ese total era poco informativo para el usuario y además enfatizaba lo largo que era el seguimiento discreto. Se simplifica el texto en pantalla para mostrar solo el paso actual ([64] ->sofa). A nivel de fluidez, no se migra al visualizador 3D de Habitat, porque eso aumentaría la complejidad sin resolver el cuello principal. En su lugar, se mantiene OpenCV y se hace más eficiente: la vista en primera persona se sigue actualizando en cada acción, pero el mapa cenital se redibuja con una cadencia menor y el retardo explícito por paso se reduce prácticamente a cero. Así, la reproducción visual resulta mucho más fluida sin penalizar perceptiblemente el rendimiento de la simulación ni complicar la arquitectura de depuración actual.
5. **Fase D implementada en el pipeline real:** el razonamiento por habitaciones deja de ser una traza auxiliar y pasa a intervenir en la decisión. Se añade una función *select_best_room()* que, a partir de *SearchState*, la evidencia del heatmap y la posición actual del robot, calcula un *room_score* por habitación con cuatro señales ya disponibles en el sistema: prior semántico, evidencia agregada del heatmap, coste BFS sobre el mapa libre y penalización por intentos fallidos previos en esa habitación. La habitación ganadora se usa inmediatamente para filtrar *kept_components* antes del ranking fino de candidatos, de modo que el flujo deja de ser puramente global. La implementación actual utiliza una proxy práctica de *unexplored* basada en visitas y fallos acumulados, y deja el seguimiento de cobertura detallada para la futura Fase F.
6. **Ajustes del flujo de búsqueda visual y preparación de tests por lote:** tras estabilizar la Fase D se corrigen varios detalles del flujo objeto→detección. Primero, la selección de goals de aproximación deja de aceptar puntos que, aun teniendo clearance y perteneciendo a la misma habitación, quedarían geométricamente al otro lado de una pared respecto al centroid del candidato; ahora se exige además línea libre sobre el mapa de planificación. Segundo, las consultas directas a muebles presentes en la escena (por ejemplo *sofa* o *sink*) dejan de pasar por el *room selector*: en ese modo se fuerza una política *nearest-first* sobre los candidatos del heatmap y el razonamiento por habitaciones se reserva para consultas indirectas o ambiguas. Tercero, cuando la query sí activa selección de habitación, se añade un *room stage* explícito: el robot navega primero a un goal interior seguro de la sala elegida y solo después realiza el acercamiento al candidato concreto, evitando verificar desde

el centro de una habitación equivocada. Cuarto, la confirmación YOLOE pasa a congelar tanto la vista RGB anotada como el punto objetivo sobre el mapa cenital, dejando una marca persistente de la detección positiva. Finalmente, se preparan scripts de *batch testing* para generar automáticamente una batería de consultas con todas las habitaciones suficientemente navegables y con todos los objetos presentes en la escena, dejando el runtime real de esas pruebas para la siguiente ronda de validación manual.

21 de abril de 2026

1. **Fase F consolidada y accesible para validación manual:** se da por cerrada la capa de acciones de alto nivel introducida el día anterior y se revisa explícitamente su alcance real. La implementación no modifica todavía el comportamiento *runtime* del robot dentro de `interactive_object_nav.py`; su función es preparar la transición desde una política *inline* a una política explícita y tipada. Quedan definidos cinco tipos de acción (`go_to_room`, `explore_room`, `inspect_candidate`, `verify_target` y `done`), su validación estructural, su serialización JSON para la futura capa LLM y un selector de *frontier* intra-habitación para `explore_room`. En paralelo, `SearchState` amplía su papel como memoria del episodio con un registro ordenado de acciones y un rastro de celdas visitadas, que servirán como base del ejecutor de la siguiente fase.
2. **Pruebas unitarias y acceso desde el menú del contenedor:** para que la validación de Fase F no dependa de Habitat, de la GPU ni de una escena concreta, se mantiene una suite `pytest` autocontenida (`tests/test_phase_f_actions.py`) y, además, se expone su ejecución desde el menú interactivo del contenedor mediante una nueva opción `p`. Esta entrada lanza la batería de pruebas de la capa de política y muestra de forma inmediata si la representación de acciones, el *round-trip* JSON, el registro extendido de `SearchState` y el *frontier picker* siguen siendo consistentes. El objetivo aquí no es medir navegación real, sino garantizar que la interfaz de alto nivel sobre la que se montará la siguiente fase permanece estable antes de conectarla al pipeline operativo.
3. **Corrección del lanzador de *batch testing*:** al ejecutar la opción `t` del menú con dataset HSSD, el script `tools/nav_batch_queries.py` fallaba con `TypeError: string indices must be integers` en `HabitatLanguageRobot.__init__`. La causa era que el script cargaba la configuración con `OmegaConf.load`, que no resuelve la lista `defaults` de Hydra: el campo `data_paths` quedaba como la cadena literal `default` en lugar del diccionario con `habitat_scene_dir/vlmaps_data_dir`. Se sustituye la carga manual por `hydra.compose` con `initialize_config_dir`, pasando los *overrides* (`scene_id`, `dataset_type`, `scene_dataset_config_f`, `data_paths`) por la vía oficial de Hydra. Tras el arreglo, el script genera `queries.txt` correctamente y el batch llega a lanzar `interactive_object_nav.py`. Queda registrado además que un `scene_id` fuera de rango (p. ej. teclear 11 cuando solo existen 0 y 1) provoca un `IndexError` en `vlmaps_data_save_dirs[scene_id]`; es una limitación de UX del menú, no un bug del pipeline, y se considerará añadir una validación previa.
4. **Primer corte funcional del ejecutor alternativo:** se materializa por fin el apartado “baseline vs. ejecutor” con una implementación inicial real. Se mantiene `interactive_object_nav.py` como *baseline* intacto y, en paralelo, se añade `application/interactive_object_nav_executor.py` como nuevo entrypoint junto con `vlmaps/policy/executor.py`. El nuevo módulo define un `ExecutorContext` y un despachador `execute_action(...)` que traduce las acciones tipadas de Fase F (`go_to_room`, `explore_room`, `inspect_candidate`, `verify_target`, `done`) a comportamiento real reutilizando las primitivas ya validadas del baseline: *room staging*, generación de *approach goals*, *path replay follower*, chequeos YOLOE en llegada y tras giro, *scan 360°* y centrado visual. La política de alto nivel se mantiene coherente con el estado del proyecto: si la query es una habitación, se genera `go_to_room` → `done`; si es un mueble

directo presente en la escena, se mantiene *nearest-first* sin selector de habitación; y para consultas ambiguas o indirectas se conserva el flujo jerárquico “habitación primero, candidato después”. Además, el menú del contenedor incorpora una nueva opción e) **Interactive executor navigation**, que permite lanzar esta variante sin tocar comandos manuales. La parte *pendiente* ya no es construir el ejecutor, sino compararlo sistemáticamente frente al baseline con las métricas de Fase H y con volcados lado a lado del heatmap baseline y del heatmap procesado *room-aware*.

5. **Resolución explícita de instancias de habitación en el parser y en la comprobación de llegada:** se corrige un fallo funcional relevante del flujo de navegación por habitaciones. Hasta este punto, órdenes como `bathroom.001` o expresiones más naturales como `bathroom 1` terminaban colapsándose sobre la habitación base `bathroom`, porque el pipeline solo distinguía familias semánticas y no instancias concretas. Se introduce un resolvidor explícito de nombres de habitación en `room_provider.py`, capaz de mapear alias amigables (`bathroom 1`, `bathroom one`, `bedroom 2`) a sus etiquetas exactas de escena (`bathroom.001`, `bedroom.002`, etc.), manteniendo a la vez el comportamiento esperado para comandos base como `bathroom` o `bedroom`, que siguen prefiriendo la habitación principal cuando existe. Esta resolución se conecta tanto al *baseline* como al nuevo *executor*: tras el parse LLM se preservan menciones explícitas de instancia presentes en la instrucción original, la selección de *room goals* pasa a usar la instancia resuelta y la condición de llegada deja de aceptar que `bathroom` satisfaga `bathroom.001`. Se añade además una batería unitaria nueva (`tests/test_room_instance_resolution.py`) que valida la distinción entre habitación base e instancia numerada, junto con la compatibilidad con el resto de la capa de acciones de Fase F.
6. **Batería de comparación baseline vs. executor disponible desde el menú:** el apartado extra deja de ser solo una propuesta en papel y pasa a tener una infraestructura operativa para lanzar comparaciones controladas. Se añade al menú del contenedor la opción v) `Compare -- baseline vs executor`, que genera un lote común de *queries*, ejecuta ambas variantes del navegador sobre ese mismo fichero de entrada y resume el resultado en CSV y markdown. La implementación queda repartida entre `tools/nav_batch_queries.py` (generación del lote), `tools/run_hssd_nav_compare.sh` (orquestación host-side) y `tools/compare_nav_runs.py` (resumen query-a-query de detecciones, habitación final, abortos y número de actualizaciones de pose). No sustituye todavía a la evaluación de Fase H, pero elimina el trabajo manual repetitivo de abrir y comparar logs a mano.
7. **Primer corte funcional de Fase G sobre el executor:** se añade `vlmaps/policy/strategic_policy.py` como capa de decisión estratégica sobre estado estructurado. Esta pieza construye un *snap-shot* del episodio (prior de habitaciones, evidencia del heatmap, shortlist de candidatos, frontiers sugeridos y últimas acciones), consulta opcionalmente a un LLM para pedir la siguiente **Action** en formato JSON y valida esa propuesta antes de ejecutarla. Si el LLM no está disponible o devuelve una acción inconsistente, el sistema cae automáticamente a una política heurística determinista, lo que evita romper el flujo ya validado del executor. A nivel de integración, `interactive_object_nav_executor.py` deja de preconstruir un plan fijo y pasa a operar como bucle “elegir siguiente acción → ejecutar acción”, con modo de política seleccionable desde el menú (`heuristic`, `hybrid`, `llm`). La validación unitaria queda cubierta por `tests/test_phase_g_strategic_policy.py`.

22 de abril de 2026

1. **El testing deja de ser una colección de utilidades y pasa a ser un flujo único:** se reorganiza por completo el submenú t del contenedor para que muestre solo tres opciones visibles: `Generate test set`, `Compare full 2x2 pipeline` y `Heatmap-only offline analysis`.

Las antiguas utilidades intermedias (batch baseline aislado, comparación binaria baseline vs. executor, runner JSONL suelto y tests unitarios de F/G) siguen existiendo como piezas internas o de desarrollo, pero dejan de ser el flujo principal del usuario. La intención es que la evaluación pase a ser reproducible y autoexplicativa: elegir escena(s), pulsar, esperar y recoger resultados en una carpeta estándar.

2. **Selección explícita del tipo de heatmap en runtime:** hasta esta fecha, baseline y executor compartían implícitamente `compute_heatmap()`, lo que impedía montar una comparación 2x2 limpia sin duplicar entypoints. Se introduce `VLMAPS_HEATMAP_MODE` con dos modos explícitos: `baseline` (heatmap crudo tras proyección 2D + decay, sin postproceso) y `postprocessed` (pipeline actual con `postprocess_heatmap`). El baseline y el executor imprimen el modo activo al arrancar y el valor queda recogido en el `manifest.json` de cada corrida. Además, ambos entypoints emiten ahora al final de cada query un resumen estructurado `[eval-summary] {...}` con habitación final, historial de habitaciones, número de candidatos inspeccionados/confirmados, rastro de acciones y recuento de celdas visitadas. Esta capa permite calcular métricas de Fase H sin depender de *regex* frágiles sobre logs textuales.
3. **Runner online 2x2 y agregador de métricas completos:** se añaden dos herramientas nuevas. La primera, `tools/run_full_2x2_eval.py`, coordina de forma automática las cuatro combinaciones `Ob_Hb`, `Oe_Hb`, `Ob_Hp` y `Oe_Hp`, reutilizando `tools/run_nav_eval.py` como runner por método. La segunda, `tools/aggregate_full_2x2_eval.py`, consume los cuatro manifiestos y genera dos vistas listas para inspección: `compare_full.csv/.md` (una fila por query) y `aggregate_metrics.csv/.md` (una fila por método y *slice*). Las salidas se agrupan bajo una estructura homogénea: `/workspace/results/eval_runs/{timestamp}/queries/, pipeline_2x2/{scene}/Ob_Hb, ...`, junto con un `config.json` raíz que describe escenas, semilla, política del executor, rutas de queries y métodos corridos.
4. **Métricas online ya instrumentadas y *slicing* por query_type/tag:** el agregador 2x2 calcula ya de forma reproducible SR, Object SR, Room SR, CFR, CT2R, Rooms Before Success, Inspections Before Success, Wrong Visits, Mean Pose Updates, Early Stop Rate, Final Room Accuracy, Preview No Action Rate, Path Stopped Early Rate, Mean Executor Actions y Mean Room Transitions. No todas las métricas tienen aún el mismo grado de robustez: por ejemplo, `Inspections Before Success` se aproxima como `candidates_tried - candidates_confirmed` cuando la query termina en éxito, y `Rooms Before Success` se interpreta como el índice de la primera habitación esperada dentro de `visit_history`. Estas aproximaciones quedan explícitas en el `Markdown` generado, en vez de presentarlas como magnitudes más precisas de lo que el log realmente permite.
5. **La evaluación offline de heatmaps gana agregación por slices y envoltorio estándar:** `tools/eval_heatmap_postprocess.py` mantiene la salida `per_query.csv`, pero añade un `aggregate_by_slice.csv` y tablas adicionales en `summary.md` para resumir el rendimiento del heatmap raw y del heatmap postprocesado por `query_type` y por `tag`. Para que esta tercera opción del submenú no requiera rutas ni carpetas manuales, se añade `tools/run_heatmap_offline_eval.py`, que normaliza la salida bajo `/workspace/results/eval_runs/{t` y escribe también su propio `config.json`.
6. **Validación rápida sin navegación larga:** se ejecutan únicamente comprobaciones ligeras, coherentes con el criterio del proyecto de no lanzar desde el asistente corridas largas de simulación sin supervisión del usuario. Han pasado `bash -n` sobre el menú, `py_compile` de los scripts y entypoints tocados, una batería `pytest` nueva con manifests sintéticos para el agregador 2x2 (`tests/test_eval_2x2_tools.py`) y, además, las suites ya existentes de Fase F, Fase G y resolución de instancias de habitación (26 tests). Con ello queda validado el armazón de evaluación; lo que falta a partir de ahora es usarlo sobre escenas y JSONLs

reales para producir resultados y discutirlos en la memoria.

23 de abril de 2026

1. **Limpieza del dataset HSSD:** se reduce el listado de escenas disponibles a las tres que se usarán en la evaluación oficial: 102344193_0, 102344280_0 y 108736884_177263634_4. Las restantes (variantes y dos carpetas con nombres corruptos generadas por capturas accidentales de `stderr` durante el `collect`) se eliminan de `data/vlmaps_dataset_hssd/`. El menú interactivo refleja ya sólo `scene_id` 0/1/2.
2. **Reversión de la rama room-aware en el harness de evaluación:** tras un día con seis métodos (orquestador $\times$ heatmap $\times$ room-aware) se decide volver al 2×2 original (Ob_Hb, Ob_Hp, Oe_Hb, Oe_Hp). Razón: el switch room-aware no es una variable independiente de evaluación sino una decisión arquitectónica del executor, y mantenerlo como eje del harness multiplicaba el coste sin aportar comparaciones nuevas para el nivel actual (muebles y habitaciones). `tools/eval_methods.py` y `tools/run_full_eval.py` vuelven a la matriz de cuatro celdas.
3. **Calibración del umbral de confianza de YOLOE:** se introduce un sweep dedicado al verificador visual sin barrer los cuatro métodos del 2×2 . Justificación: el `conf-thresh` sólo afecta la fase de verificación al llegar al candidato; las decisiones de navegación, heatmap y selección de habitación son independientes. Por tanto basta con barrer un único método (Oe_Hp) para aislar el efecto del umbral.
4. **Nueva herramienta `tools/run_yoloe_thresh_sweep.py`** (encargada a Codex). Corre `run_nav_eval.py` repetidamente sobre la misma escena y JSONL, una vez por `threshold`, propagando el valor vía `VLMAPS_YOLOE_CONF_THRESH`. Al terminar, agrega los manifests y produce `compare_thresh.csv` y `compare_thresh.md` con una fila por `threshold` y columnas SR, Object SR, Found rate, Wrong visits, CFR, CT2R, Early Stop Rate. Convive sin tocar el harness 2×2 ni el submódulo `vlmaps`.
5. **Resultado del barrido sobre $\{0,30, 0,40, 0,50, 0,60\}$** (escena 102344193_0, 50 queries, método Oe_Hp): $\tau = 0,30$ maximiza SR (**0.82**), Object SR (**0.76**) y *found rate* (**0.62**) y minimiza las visitas erróneas (**0.84**). A partir de $\tau = 0,40$ la degradación es monótona: las visitas erróneas se triplican y la SR cae 16 puntos en $\tau = 0,60$. La constancia de CFR (0,18 en los cuatro runs) confirma que el experimento aísla correctamente el verificador y no contamina la navegación. Los detalles cuantitativos y la decisión están integrados en la subsección *Calibración del umbral de confianza de YOLOE* del bloque de testing.
6. **Discusión honesta sobre la medición de falsos positivos:** durante el análisis se descarta el indicador *FP-proxy* inicial (`found` $\wedge$ $\neg$ `success` sobre queries `room_object`). Razones: (i) el *ground truth* del *benchmark* es a nivel de habitación, no de instancia, por lo que encontrar una instancia válida del mismo objeto en otra habitación se contabiliza erróneamente como falso positivo; (ii) el agente detiene la búsqueda en la primera confirmación, ocultando posibles FPs latentes en otras habitaciones; (iii) con sólo 8 queries `room_object` el indicador es estadísticamente inservible. Se documenta como limitación del banco de evaluación que una medición directa de FPs requeriría *queries negativas* (objetivo ausente en la escena) o *ground-truth* de instancia.
7. **Decisión adoptada:** se mantiene $\tau = 0,30$ como umbral de confianza para todos los métodos del 2×2 oficial. La elección queda justificada documentalmente (tabla, gráficos y discusión) en la sección de testing, evitando aparecer como un default arbitrario.
8. **Decisión estratégica: activar la migración a ROS 1 Noetic antes que el nivel 3.** Tras revisar el coste real de añadir nivel 3 (objetos pequeños) directamente sobre Habitat y luego portarlo a ROS, se concluye que esa secuencia obliga a iterar dos veces sobre los

mismos componentes (Phase G, surrogate furniture, YOLOE bridge, harness). Se reordena el roadmap para migrar primero el stack a ROS 1 Noetic + Gazebo y desarrollar el nivel 3 directamente sobre el stack final. Plazo asumido para la migración completa: **un mes** (hasta el 23 de mayo de 2026).

9. **Renuncia explícita a HSSD como dependencia obligatoria:** las escenas pasarán a ser *house worlds* de Gazebo (HSR oficial, AWS RoboMaker) o iGibson (15 apartamentos basados en escaneos reales, utilizado por MoMa-LLM). El razonamiento por habitaciones, surrogate furniture y verificación con YOLOE son agnósticos a la fuente de la escena.
10. **Esquema de etiquetado de habitaciones por centroides + Voronoi** (estilo MoMa-LLM): se sustituye el sistema de polígonos `semantic_config.json` por una partición de Voronoi sobre centroides marcados manualmente (un *click* por habitación). El mismo esquema vale para escenas Gazebo, iGibson y, más adelante, para el laboratorio físico. Coste de anotación: 5–10 minutos por escena. Implementación en `room_provider` estimada en una sesión manteniendo la interfaz pública.
11. **Inspiración explícita en MoMa-LLM** (Honerkamp et al., RSS SemRob 2024): adopción conceptual de tres ideas del paper, con re-implementación propia y *sin copiar código* (compromiso de integridad académica). Las ideas adoptadas son: (i) clustering de habitaciones por Voronoi; (ii) vocabulario de acciones de alto nivel (ya parcialmente cubierto por la Phase F existente); (iii) métrica AUC-E (área bajo la curva de eficiencia) como complemento al SR a *budget* fijo, para reportar resultados en la memoria.
12. **Continuidad de Docker en host distinto:** la migración mantiene Docker como mecanismo de empaquetado. El contenedor pasa de `tfg-sim` (Habitat-Sim sobre Ubuntu 20.04) a un nuevo contenedor Noetic+Gazebo sobre Ubuntu 20.04. El usuario sigue trabajando desde su host Ubuntu 22.04 sin reinstalar nada en el host. Cuando se traslade el proyecto a un equipo nativo Ubuntu 20.04, el contenedor sigue siendo válido (ROS nativo en el host se considera optimización opcional, no requisito).